Progetto PISSIR

Introduzione

L'obiettivo del progetto è simulare la realtà di un'azienda che installa delle macchinette per la distribuzione di bevande calde in vari istituti scolastici, gestendo sia la parte amministrativa sia le singole macchinette.

Specifica

Fase di specifica: Macchinetta

Ogni macchinetta:

- deve tener traccia di quante cialde ha per bevanda e zucchero, in modo da segnalare quando è necessario chiamare il tecnico (se la macchinetta è vuota);
- deve mostrare le bevande che possono essere erogate su uno schermo (nel nostro caso verranno mostrate quelle le cui cialde sono disponibili);
- deve inoltre tenere traccia dei bicchieri disponibili.
- deve tenere il denaro inserito in una cassa temporanea fino all'erogazione effettiva della bevanda, in modo da poter restituire i soldi all'utente nel caso li richieda indietro;
- deve poter restituire il resto in seguito a un acquisto (se le monete sono disponibili);
- deve poter controllare che le monete inserite siano valide;
- deve controllare se il prezzo necessario per la bevanda selezionata sia stato raggiunto o meno;
- deve possedere una cassa 'definitiva' con tutti gli incassi in seguito agli acquisti, la quale verrà svuotata dal tecnico quando piena.
    - Nel momento in cui la cassa è piena, la macchinetta diventa non utilizzabile e viene chiamato il tecnico automaticamente;
- deve poter contattare il tecnico in caso di:
    - cassa piena;
    - cialde non disponibili (macchinetta vuota);
    - guasto generico in una qualsiasi delle componenti;
- deve essere identificata da un id univoco, in modo che sia rintracciabile facilmente.

Bevande, cialde e monete possibili

1. Bevande disponibili:
    a. caffé (1 cialda per bicchiere)
    b. latte caldo (1 cialda), thé (1 cialda)
    c. mocaccino (1 caffé, 1 latte, 1 cioccolata)
    d. caffé macchiato (1 cialda di caffè e 1 di latte)
    e. cioccolata fondente (1 cialda)
    f. cioccolata al latte (1 cioccolata, 1 latte)
2. Cialde possibili:
    a. latte
    b. caffé
    c. cioccolato
    d. thé
3. Monete utilizzabili:
    a. 5 cent
    b. 10 cent
    c. 20 cent
    d. 50 cent
    e. 1 euro
    f. 2 euro

Storie e scenari

Un utente vuole utilizzare una macchinetta, quindi verifica che la cassa sia utilizzabile.

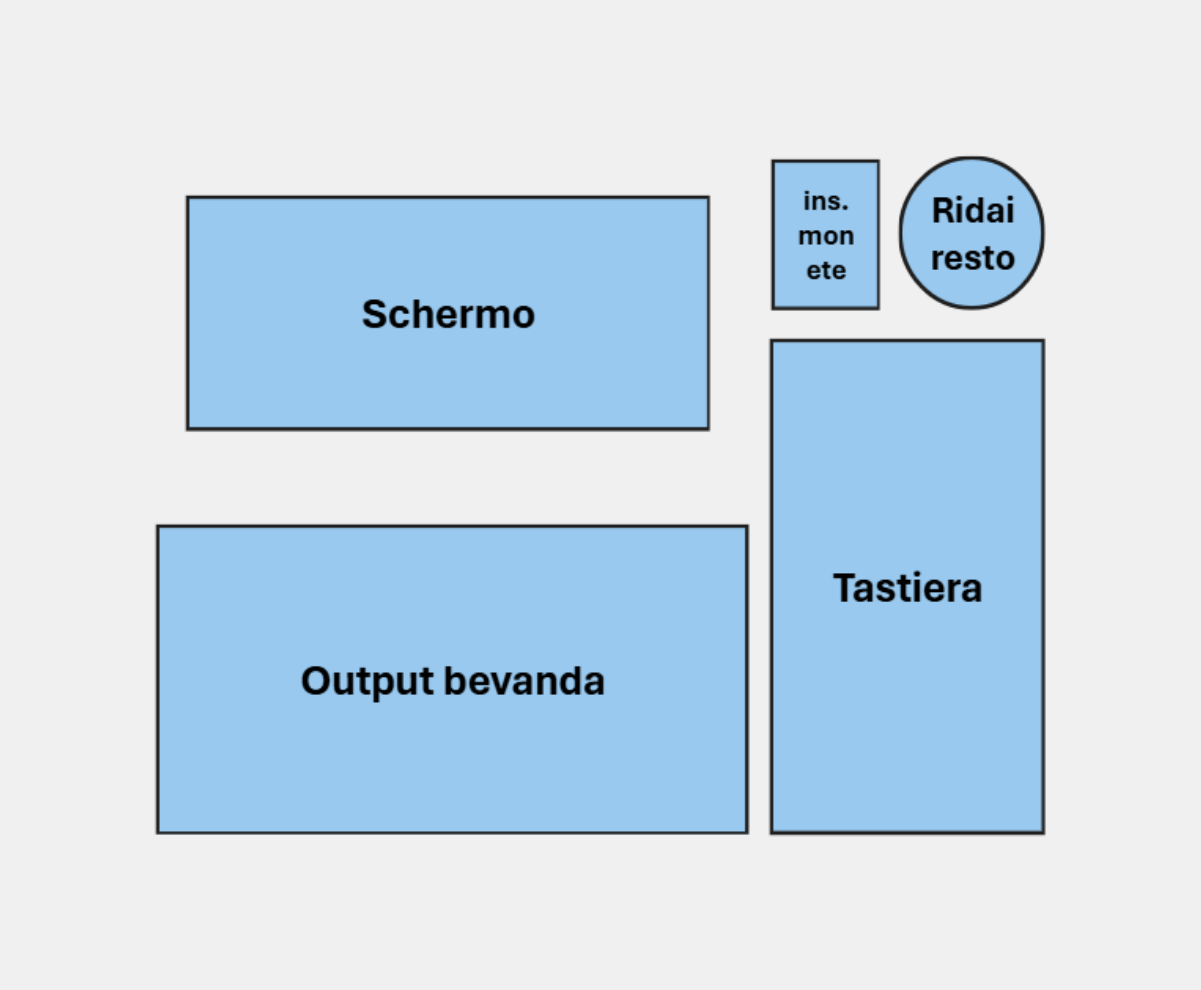
1. **Se i controlli vanno a buon fine**:
    a. la macchinetta invia al server rest il resoconto in più messaggi (specificando l'id), dicendo:
        i. quali cialde mancano e quali si stanno esaurendo
        ii. se la cassa è piena o vuota
    b. mostra le bevande disponibili (per fare subito controllo cialde disponibili)
    c. A questo punto, l'utente può effettuare alcune azioni:
        i. **selezionare la bevanda**: se la cassa è vuota o il credito inserito non è sufficiente, il prezzo appare sullo schermo ma non viene effettuato alcun acquisto (se il credito non è sufficiente comparirà un messaggio "Credito non sufficiente");
        ii. **inserire una moneta** (può essere fatto più volte, ogni volta controllando cassa piena o vuota);
        iii. **richiedere indietro i suoi soldi** cliccando l'apposito bottone rosso: la cassa temporanea verrà svuotata;

iv. Se l'utente seleziona una bevanda e nella cassa temporanea è stata raggiunta la somma necessaria:
    1. prezzo corretto, no resto: erogazione bevanda, l'utente preleva bevanda, fine.
    2. prezzo raggiunto, si resto: erogazione bevanda, calcolo resto, l'utente preleva bevanda, fine.

2. **Se i controlli non vanno a buon fine** oppure in un qualunque momento in cui si verifica un **guasto in una componente**: comunicazione al tecnico della necessità di manutenzione e momentanea disabilitazione della macchinetta.

Idea iniziale: UI macchinetta

L'interfaccia utente deve contenere queste parti per garantire un corretto utilizzo da parte dell'utente:



Fase di specifica: Livello Gestionale

A livello gestionale:

- sono necessarie varie tabelle per poter gestire N macchinette distribuite su M istituti (detti anche scuole), tenendo traccia degli id di ciascuna macchinetta.
- Se si è impiegati:
    - è possibile visualizzare:
        - l'elenco delle macchinette totali
        - l'elenco delle macchinette per istituto
        - l'elenco degli istituti
- Se si è amministratori:
    - è possibile visualizzare:
        - l'elenco delle macchinette totali
        - l'elenco delle macchinette per istituto
        - l'elenco degli istituti
    - è possibile aggiungere o eliminare una macchinetta;
    - è possibile aggiungere o eliminare un istituto all'elenco di quelli coinvolti;
- chiamata del tecnico: ogni segnalazione di problema viene salvata in una tabella, con vari dettagli inerenti al tipo di guasto da riparare / manutenzione necessaria. Da questa tabella, il tecnico potrà vedere il necessario per risolvere i problemi e poi, alla fine, segnare ciò che ha risolto, in modo da riabilitare l'utilizzo della macchinetta;
    - se la tabella è vuota, non è possibile chiamare il tecnico (non viene avvisato);
    - se nella tabella ci sono richieste, il tecnico controlla la causa del guasto per una richiesta e va in loco per risolverlo;
- il denaro dalle varie casse viene recuperato (ogni volta che la cassa di una macchinetta è piena) e gestito.

Storie e scenari: Livello gestionale

1. Mostrare la situazione attuale (quante macchinette, cialde, ecc)
2. mostrare al tecnico le richieste di guasto delle varie macchinette:
    - la cassa della macchinetta è piena e va svuotata
    - mancano delle cialde o sono in esaurimento
    - manca lo zucchero o è in esaurimento
    - mancano i bicchierini o sono in esaurimento
    - c'è stato un guasto generico
3. aggiungi/rimuovi una macchinetta in un istituto (eventualmente creando/rimuovendo anche l'istituto) (solo per amministratori)

Diagramma dei casi d'uso

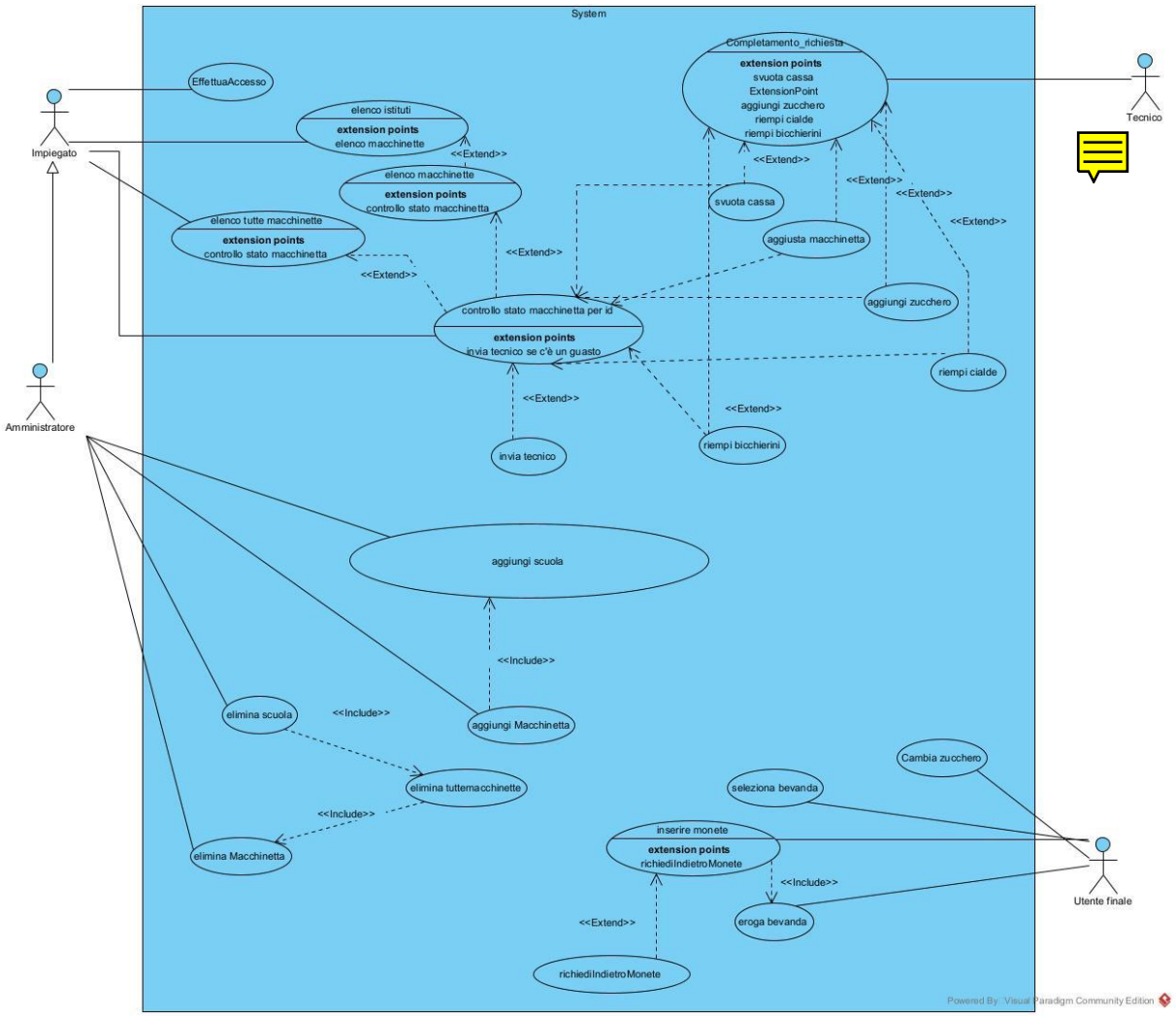
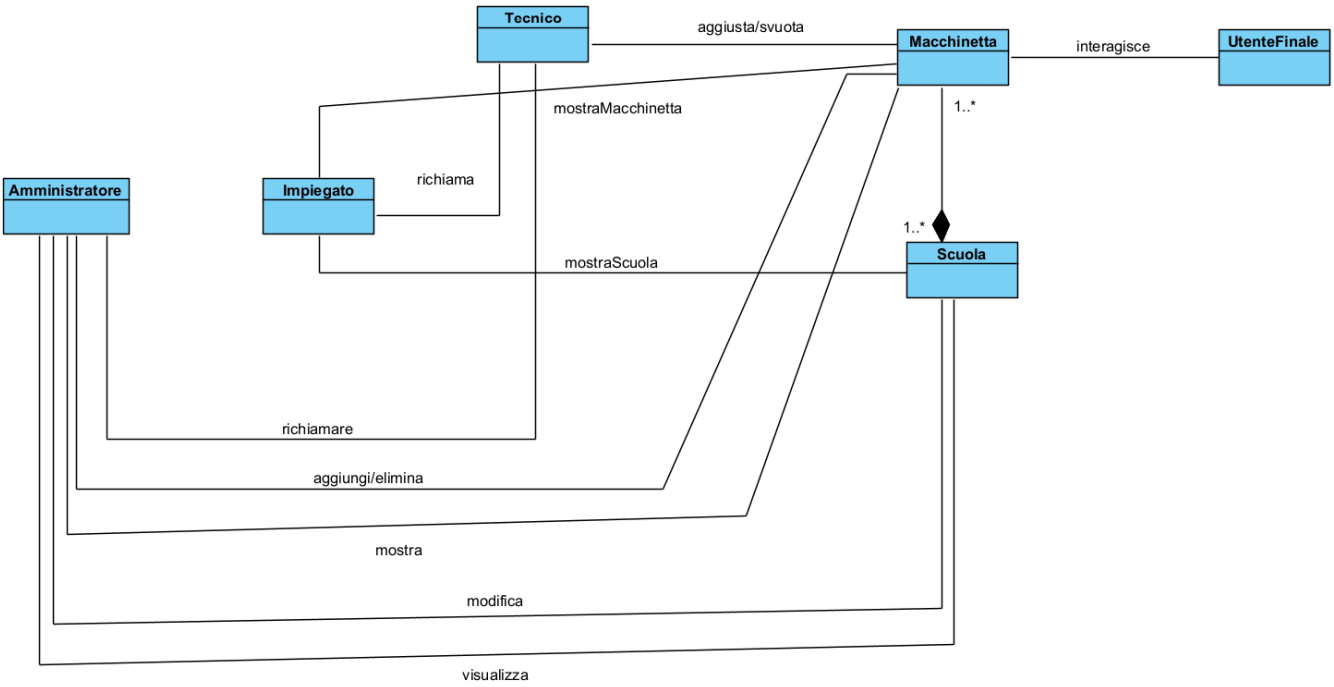


Diagramma di dominio



Progettazione

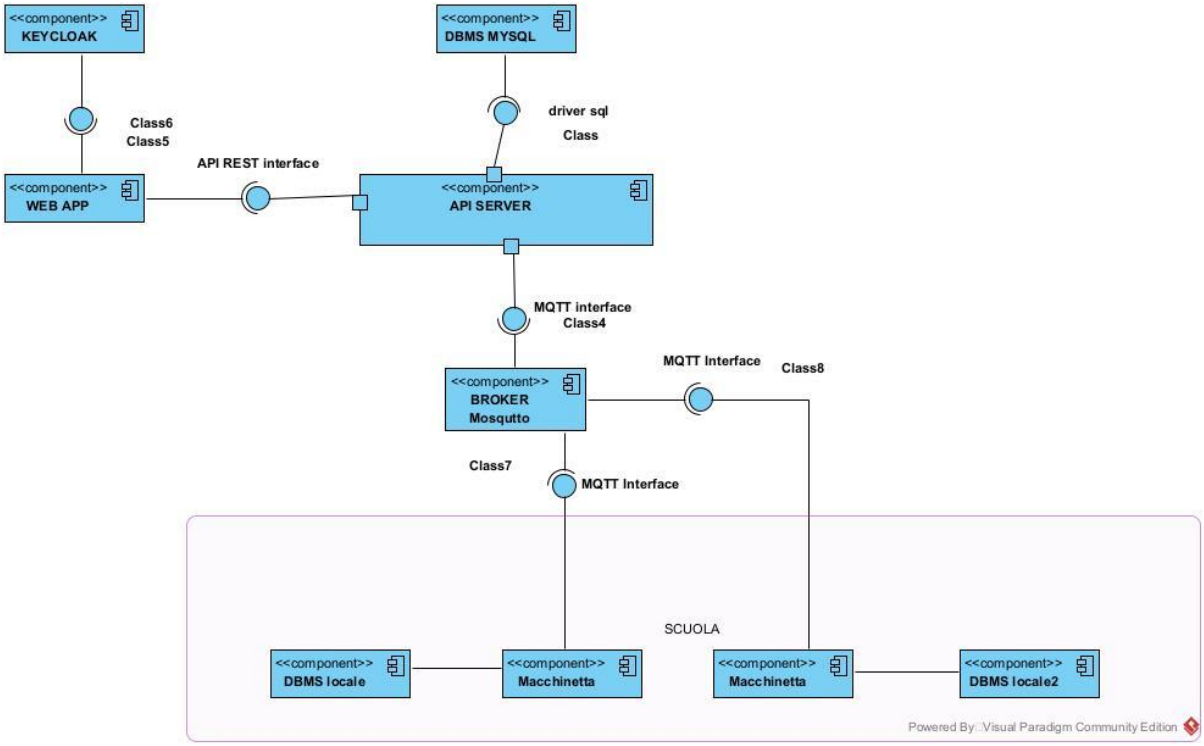
Divisione del lavoro

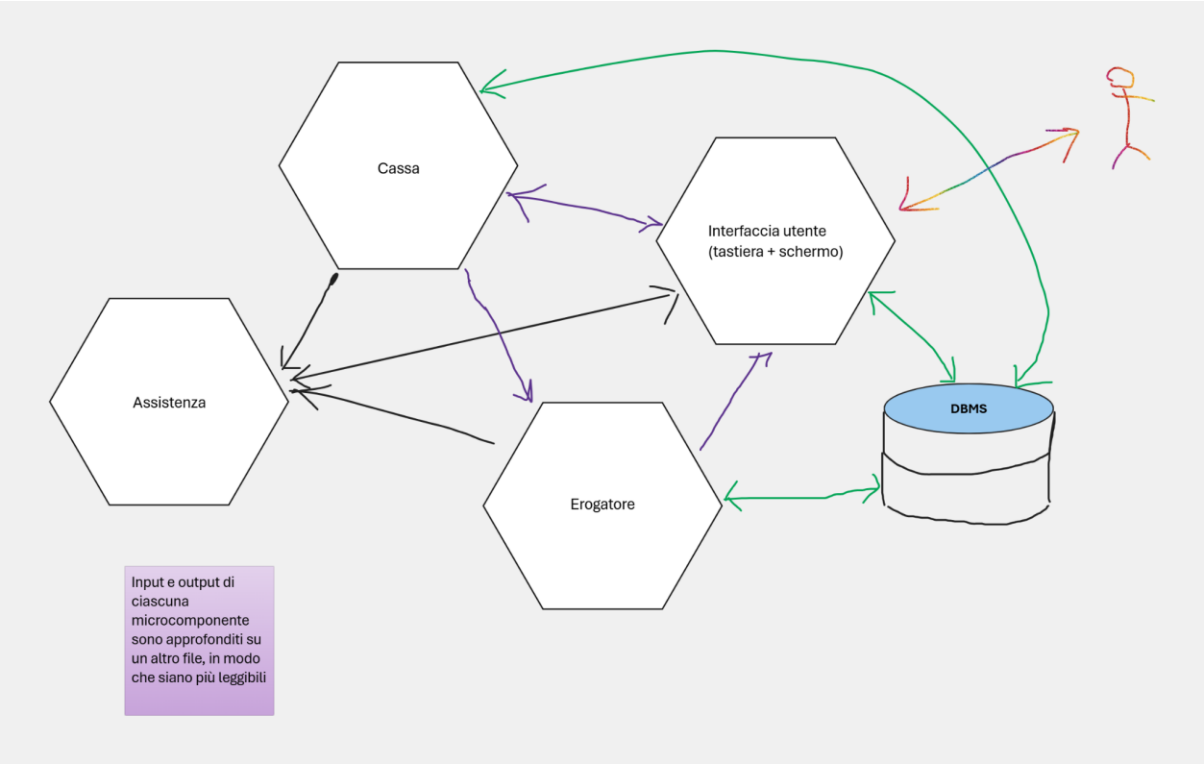
Nome della parte	**Chi se ne è occupato**
Fase di specifica	Entrambi
Fase di progettazione	Entrambi
Diagrammi	Entrambi
Stesura della documentazione	Entrambi
Implementazione Server REST	Entrambi
Utilizzo di MySQL nella parte gestionale	Entrambi
Gestione di file .json nella parte gestionale	Entrambi
Creazione e configurazione del progetto Maven (file pom.xml)	Entrambi

Implementazione della parte gestionale in lato front-end (web app) in ambiente ASP.NET (C#)	Delia Marino
Autenticazione utilizzando Keycloak nella parte gestionale (con uso di Docker)	Delia Marino
Configurazione di mosquitto (file .pwd, creazione degli utenti e specifica dei permessi relativi a ciascuno nell'acl_file.txt, …)	Alex Ruggero
Implementazione del microservizio Assistenza e relativi test (utilizzo di JUnit)	Alex Ruggero
Implementazione del microservizio Interfaccia Utente e relativi test (utilizzo di JUnit)	Alex Ruggero
Implementazione della GUI (Interfaccia Utente) con l'utilizzo di NetBeans (generando una base e poi personalizzandola in base alle esigenze)	Alex Ruggero
Implementazione del microservizio Cassa e relativi test (utilizzo di JUnit)	Delia Marino
Connessione a SQLite	Delia Marino
Gestione del file .json per i db locali di ogni macchinetta	Delia Marino
Implementazione del microservizio Erogatore e relativi test (utilizzo di JUnit)	Delia Marino
Creazione di una nuova macchinetta	Entrambi

Macchinetta

Diagramma delle componenti





Componenti e interazione tra di esse

(Tenendo conto che il controllo delle cialde disponibili per le bevande è a monte, vengono visualizzate solo le bevande disponibili)

Le componenti coinvolte sono:

- **Erogatore**: sceglie cialde, bicchierini, zucchero e controlla le cialde
- **Cassa**: gestisce le monete, il totale soldi, controllo dell'importo, gestione del resto
- **interfaccia utente**: comunica con utente per monete e bevande, segnala guasti all'utente
- **Assistenza**: segnala guasti all'interfaccia e al server

Assistenza

Input (da):

- Erogatore: mancano le cialde, mancano i bicchieri, guasto generico, manca zucchero
- Interfaccia u.: guasto generico
- Cassa: guasto generico, cassa piena

Output (a):

- Erogatore: cialde rimesse, bicchierini rimessi, guasto aggiustato, zucchero messo
- Interfaccia u.: guasto aggiustato
- Cassa: cassa svuotata, guasto aggiustato

Interfaccia Utente

Input (da):

- tastiera: inserimento soldi, numeri bevande, segnale riavere soldi, quanto zucchero mettere
- Assistenza: segnale problemi risolti
- Erogatore: bevanda pronta, bevanda consegnata, erogazione in corso

Output (a):

- schermo: stato dell'erogazione (bevanda erogata), monete restituite, prezzo della bevanda selezionata, contenuto cassa temporanea
- Cassa: numero bevanda, monete appena inserite, quanto zucchero mettere, se restituire o meno il contenuto della cassa temporanea
- Assistenza: guasto generico
- Erogatore: (nessun output)

Cassa

Input (da):

- Interfaccia u. (se la cassa non è piena): numero bevanda, monete appena inserite, zucchero (da passare all'erogatore), se deve restituire monete
- Erogatore: (nessun input)

Output (a):

- Assistenza: cassa piena, guasto generico

- Erogatore: numero bevanda da fare, zucchero da mettere
- Interfaccia: (nessun output)

Erogatore

Input (da):

- Cassa: numero bevanda da fare, zucchero
- Interfaccia: (nope)
- Assistenza: (nessun input)

Output (a):

- Cassa: (nulla)
- Interfaccia: bevanda erogata
- Assistenza: guasto generico, bicchierini in termine, cialde in termine, manca lo zucchero

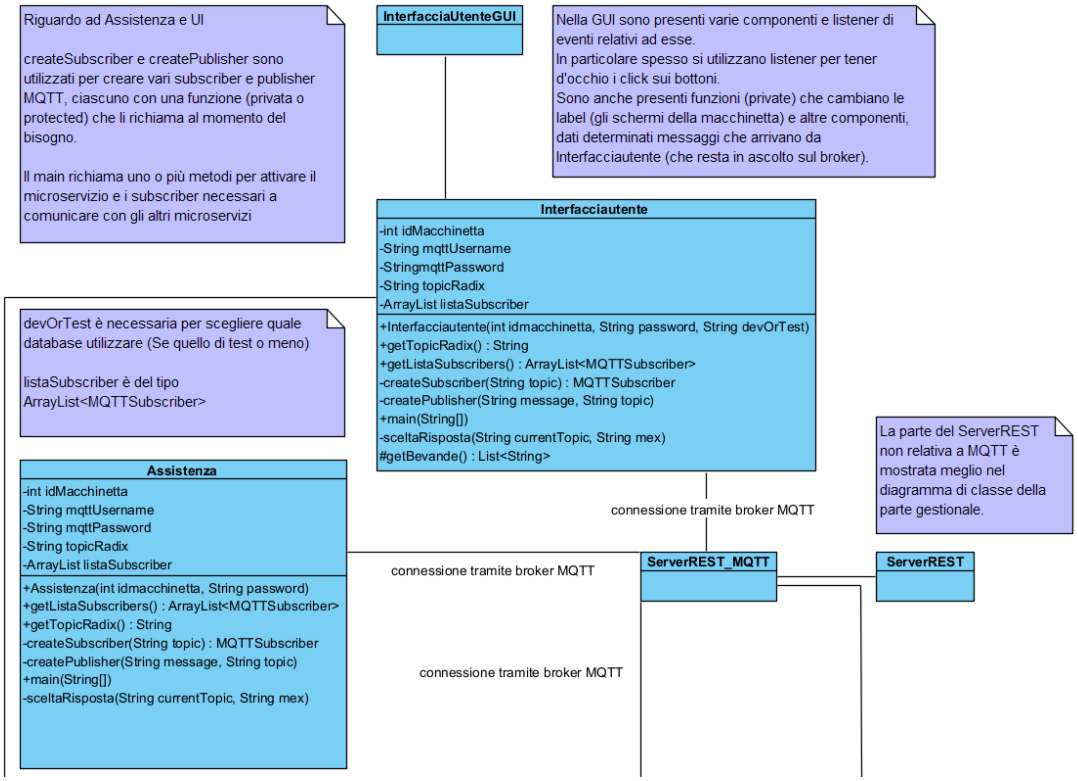
Implementazione

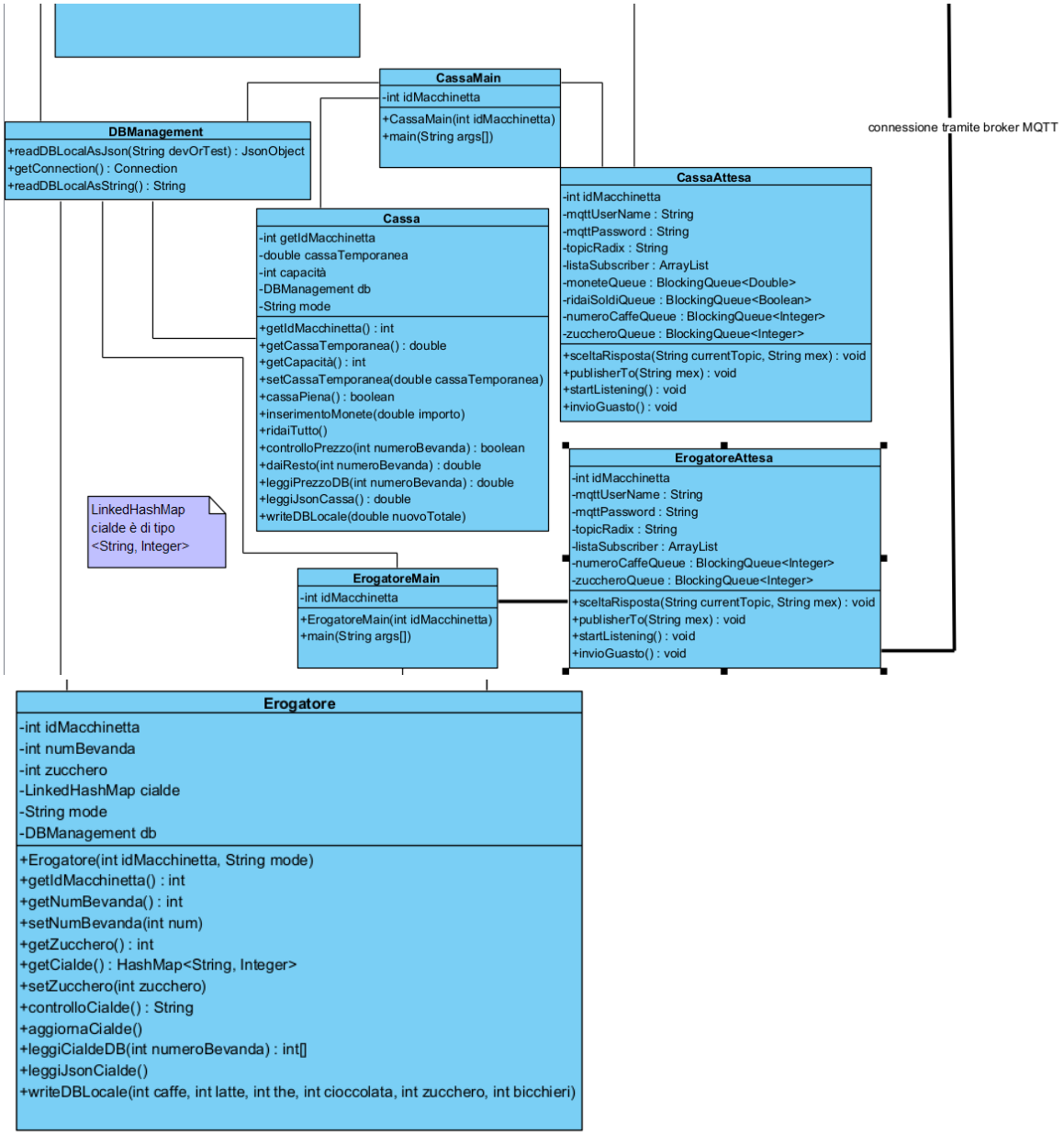
Requisiti tecnici

- Necessario windows (gestione dei path e utilizzo di visual studio per la parte web);
- necessario utilizzo di JDK 22 o più;
- necessario avere installato in locale MySQL;
- necessario avere installato Docker Desktop;
- necessario avere installato Mosquitto in locale (o su una virtual machine);

Macchinetta

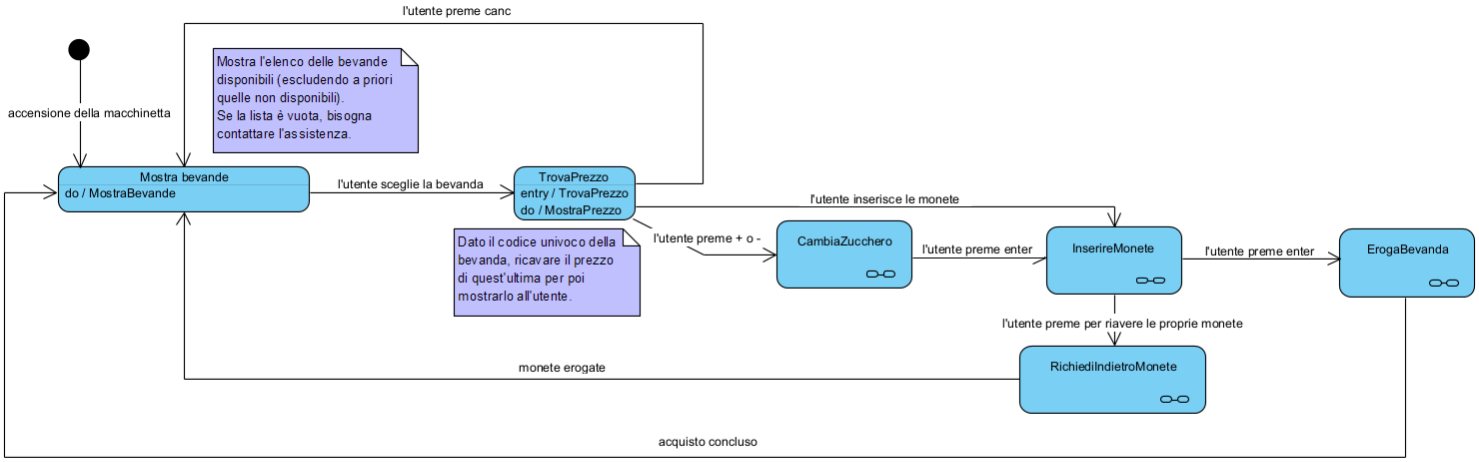
Diagrammi di classe



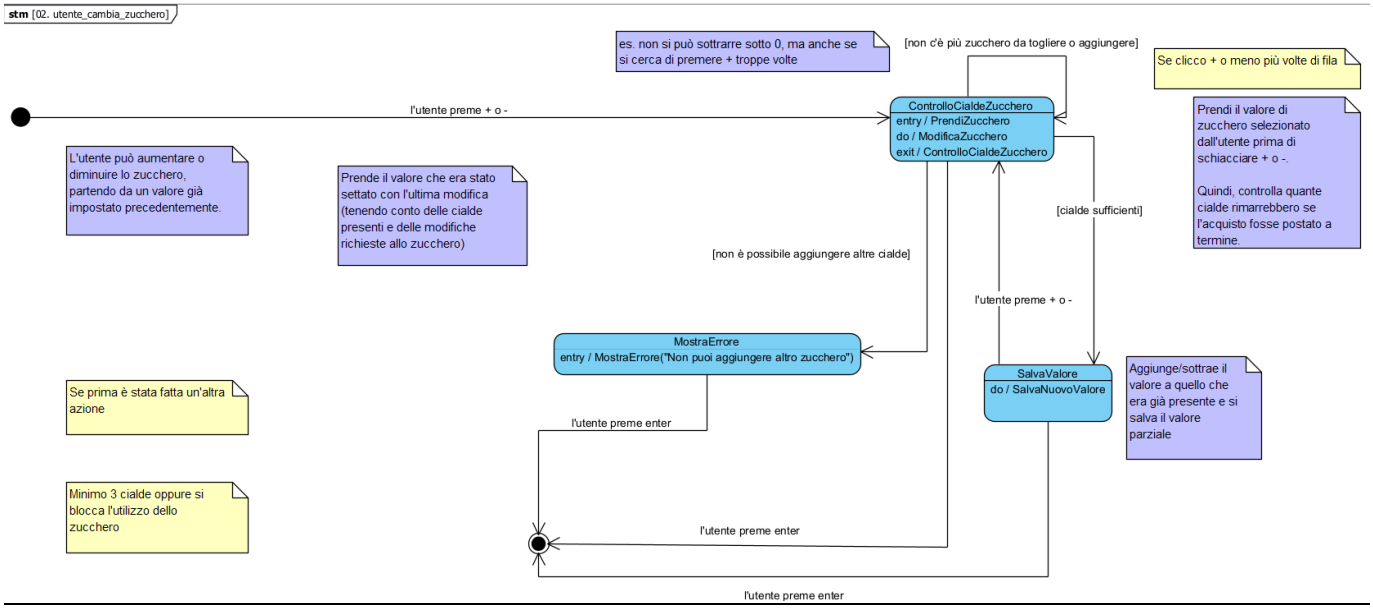


State Machine diagrams

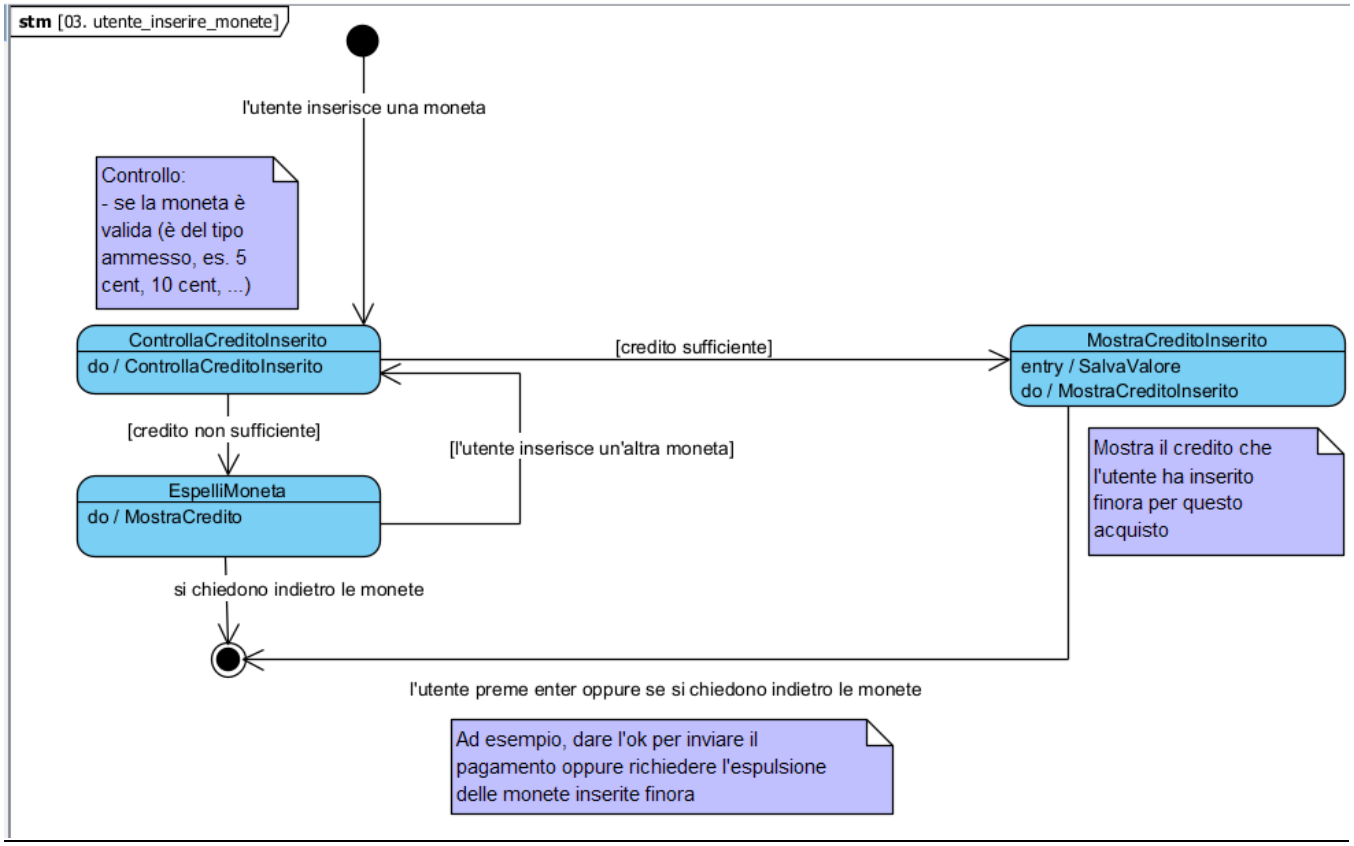
L'utente seleziona la bevanda:



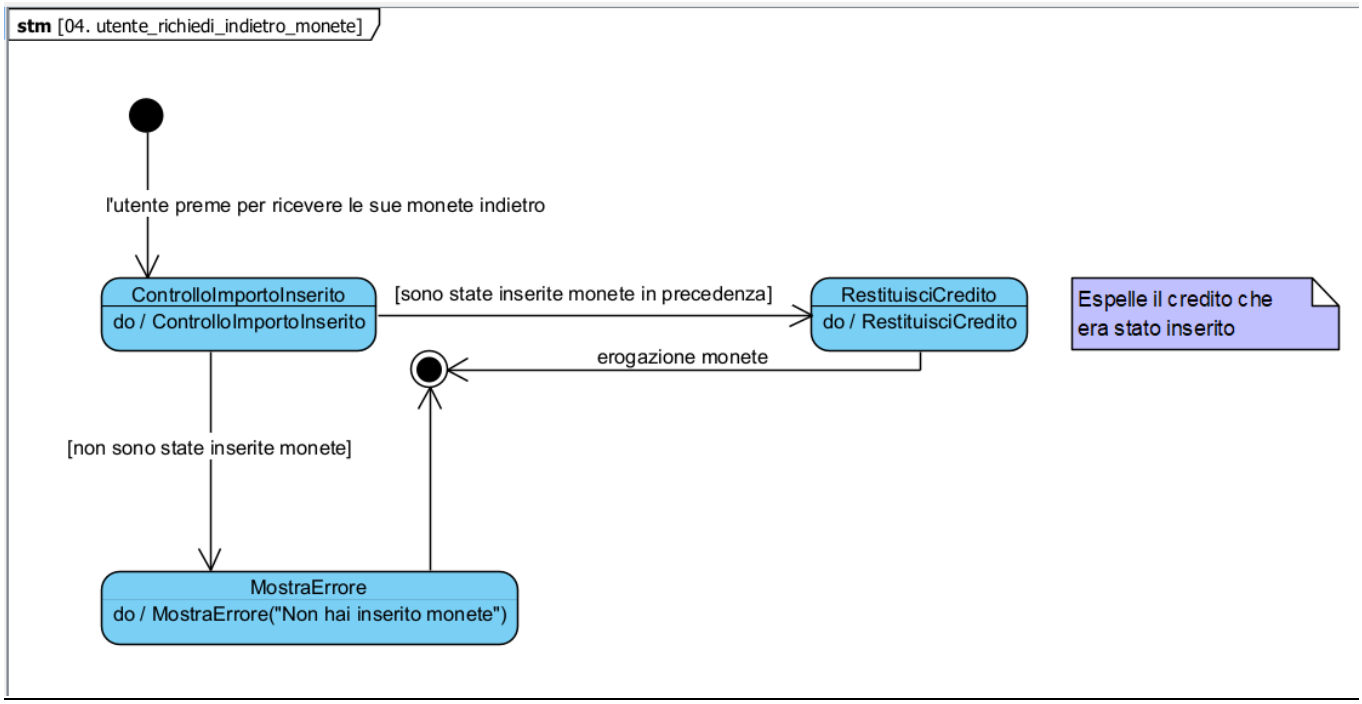
L’utente cambia zucchero:



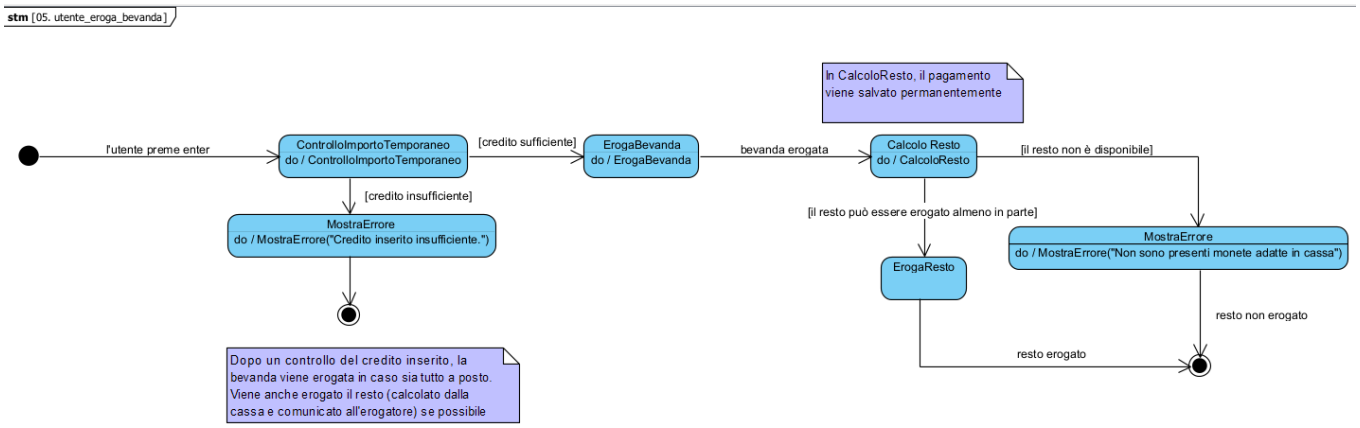
L’utente inserisce la moneta:



L’utente richiede indietro le monete:

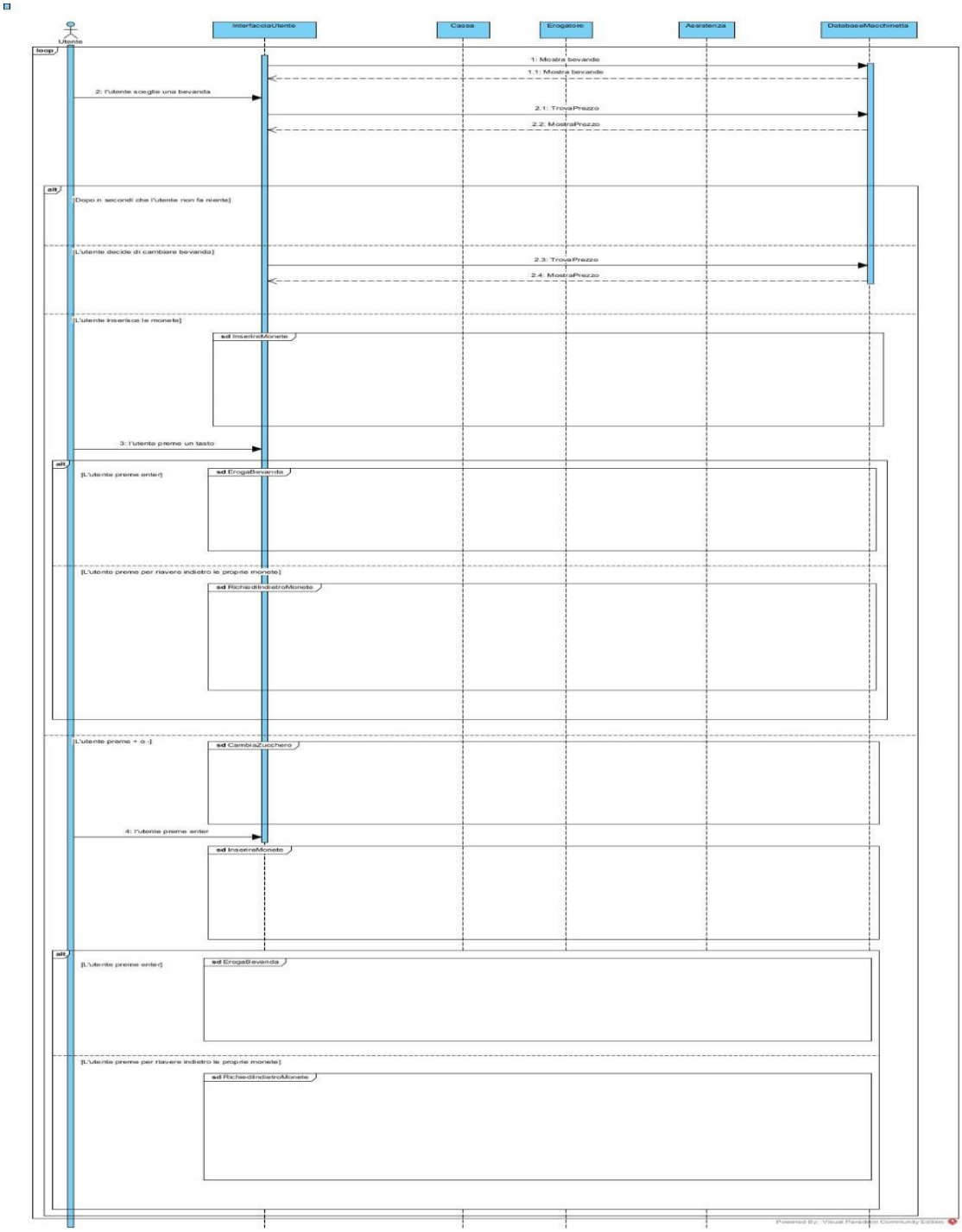


La macchinetta eroga la bevanda:

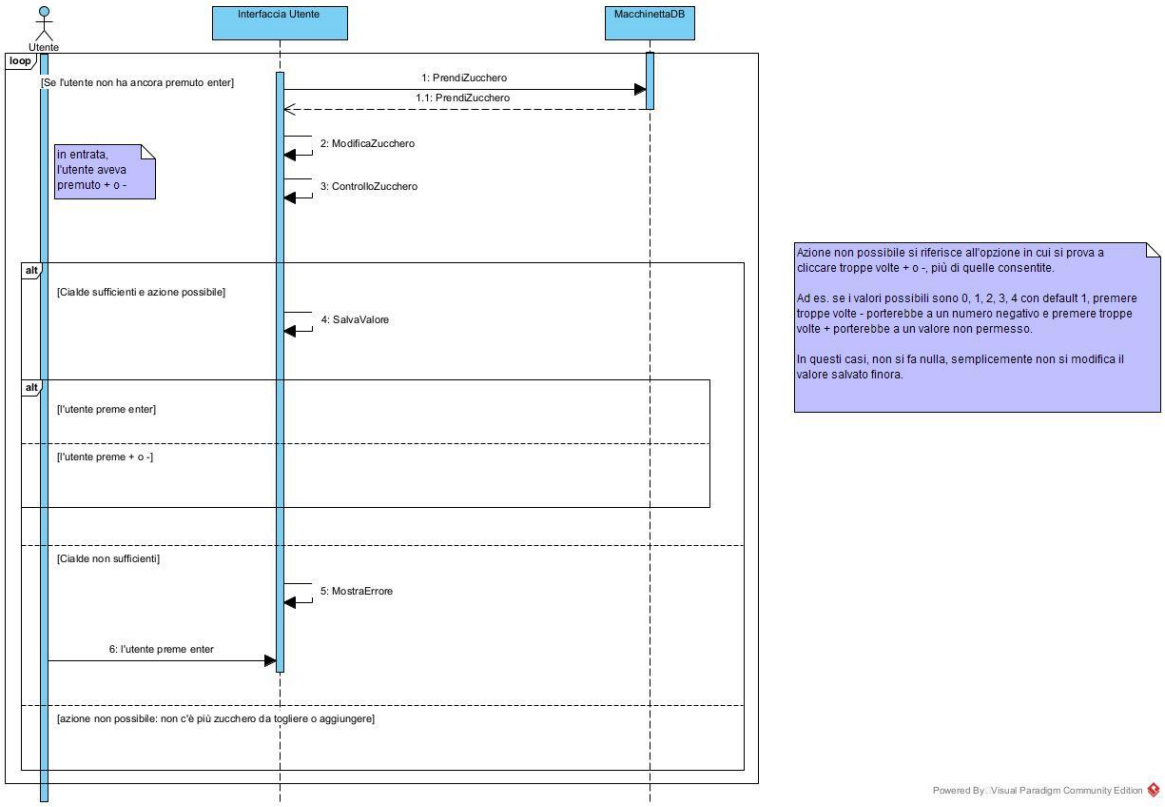


Diagrammi di sequenza

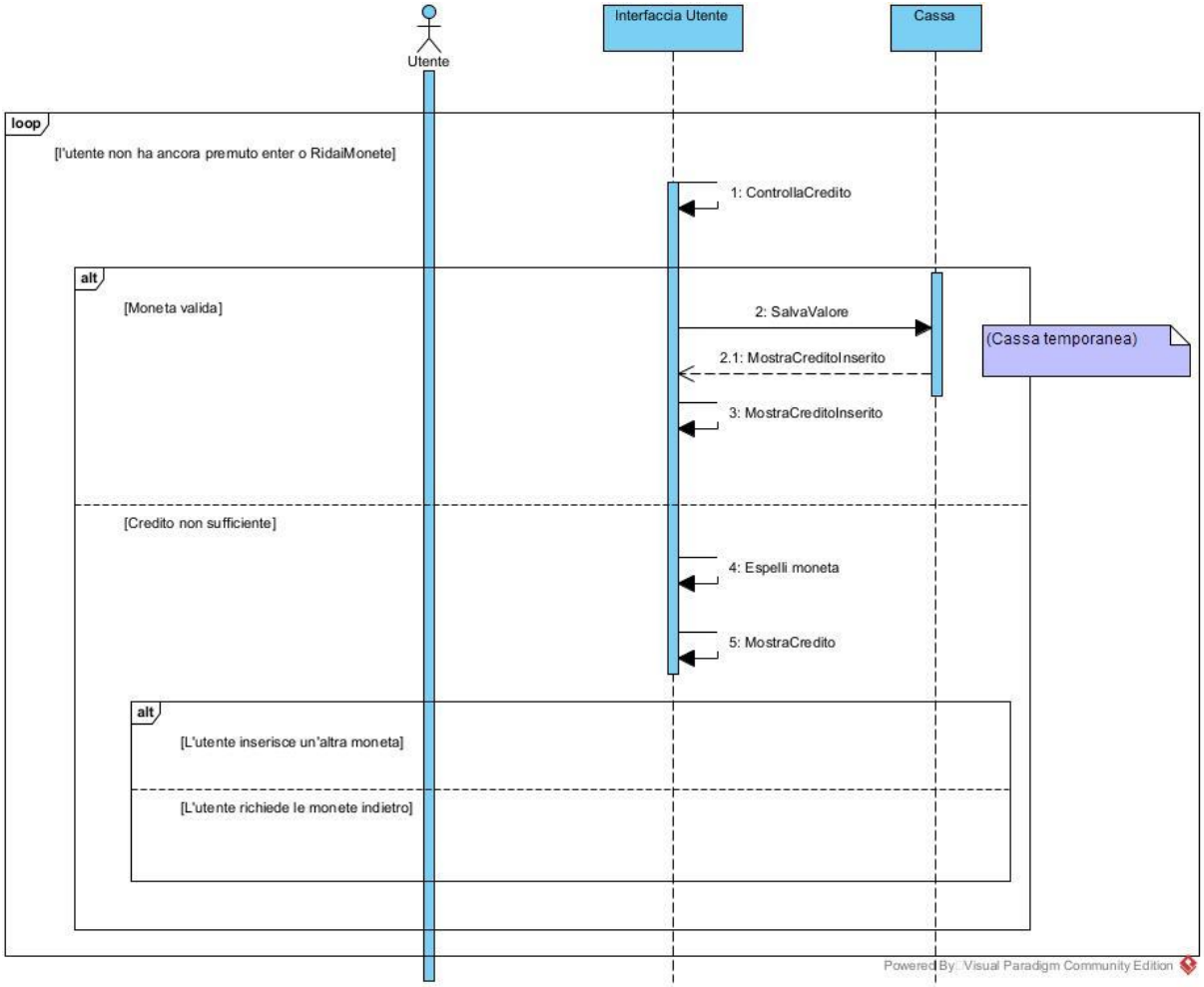
L'utente seleziona una bevanda:



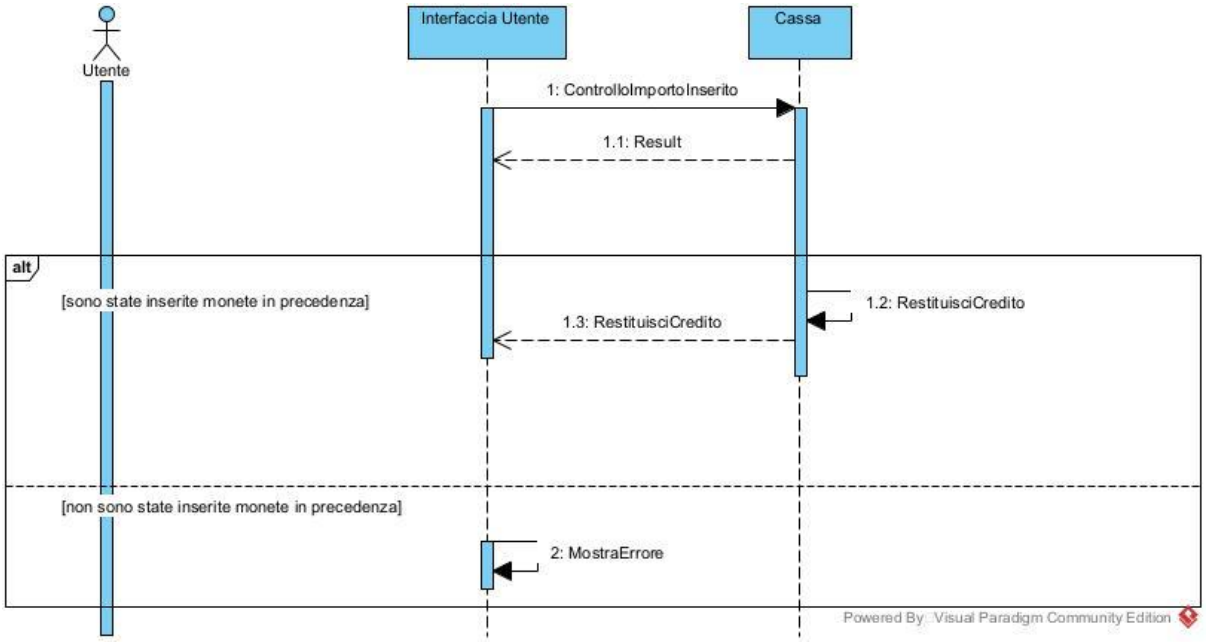
<u>L'utente cambia lo zucchero:</u>



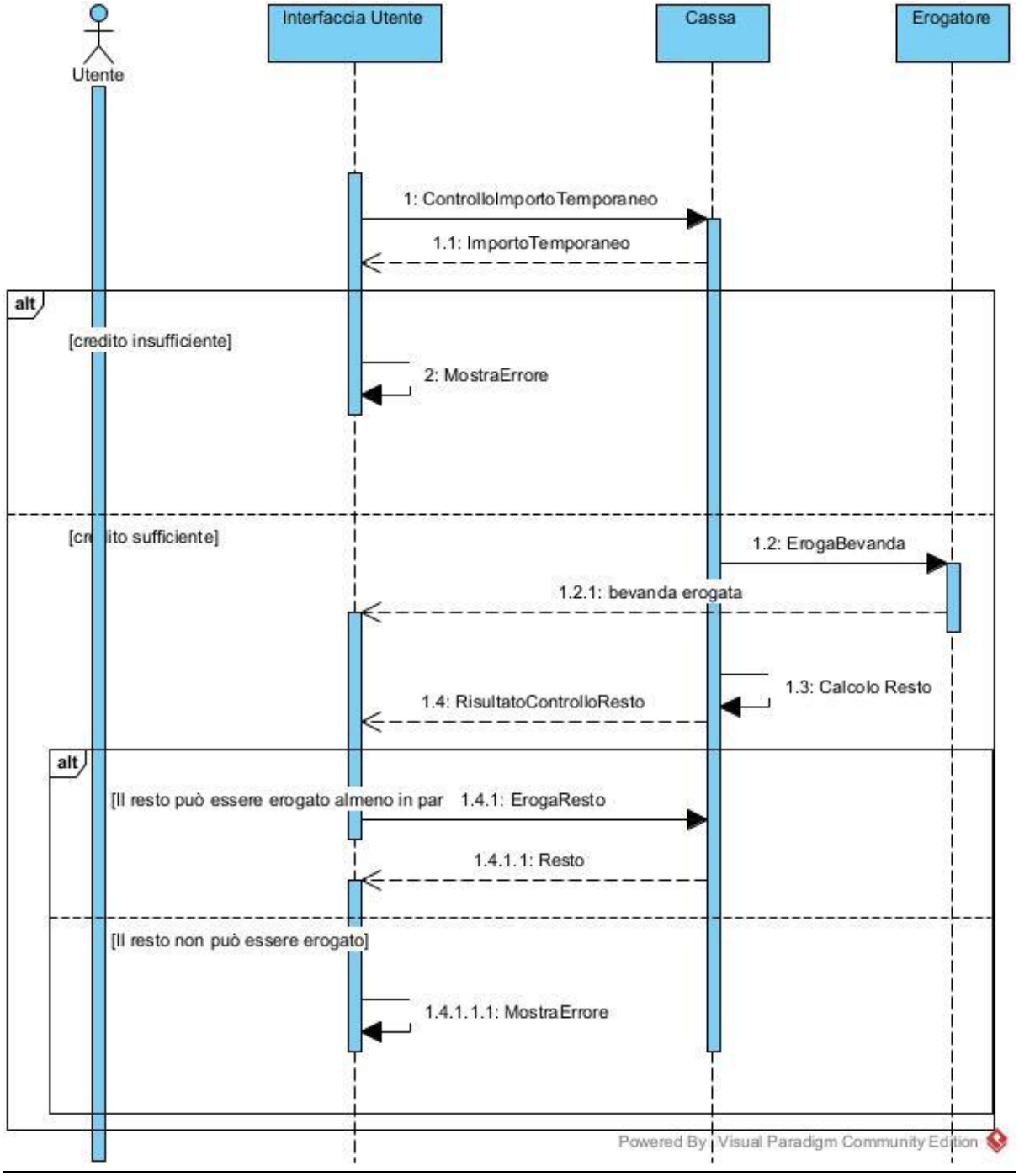
L'utente inserisce le monete:



L'utente richiede indietro i soldi:



La macchinetta eroga la bevanda:



Utenti Mosquitto

Tipi di utenti:

- cassa
- erogatore
- assistenza

- interfaccia_utente
- serverREST (ne esiste solo uno)

Ogni username è formato dal nome della componente e l'id della macchinetta (es. la cassa della macchinetta 1 sarà cassa1).
Per semplicità di implementazione, abbiamo utilizzato id semplici (contando da 1 in poi), ma in un contesto aziendale potrebbe essere necessario stabilire una struttura più complessa per gli id, ad esempio per aiutarsi con la ricerca.

Topic Mosquitto

Struttura di un topic

/macchinetteReti2/id_macchinetta/nome_microservizio_mittente/nome_microservizio_destinatario/

Messaggi sui topic Mosquitto

cassa -> assistenza -> serverREST (cassa/assistenza e assistenza/serverREST):

- "La cassa è piena"
- "Si è verificato un guasto - cassa"
- "sync json cassa-server + json"

interfaccia_utente -> assistenza -> serverREST (interfaccia_utente/assistenza e assistenza/serverREST):

- "Si è verificato un guasto - interfaccia_utente"

serverREST -> assistenza -> interfaccia_utente (serverREST/assistenza e assistenza/interfaccia_utente):

- "riparazione effettuata - interfaccia_utente"
- "problemi nella riparazione - interfaccia_utente"

assistenza/interfaccia_utente:

- "abilita gui"
- "disabilita gui"
- (i messaggi nella sezione precedente)

erogatore-> assistenza -> serverREST (erogatore/assistenza e assistenza/serverREST):

- "Si è verificato un guasto - erogatore"
- "sync json erogatore-server + json"

cassa/interfaccia_utente:

- "Questa è la moneta inserita: numero"
- "Richiesta restituzione soldi"
- "Bevanda selezionata: numero - numero Zucchero"

cassa/interfaccia_utente:

- “Il resto è numero”
- “Credito non sufficiente”
- "contenuto cassa temporanea - numero"

cassa/erogatore:

- “Eroga la bevanda numero - zucchero”

erogatore/interfaccia_utente:

- “bevanda erogata"

erogatore/assistenza:

- “Mancano le cialde del tipo: cialda”

Simulazione di guasti

Sono presenti delle funzioni per simulare situazioni di guasti nelle varie componenti e sono gestiti direttamente da ogni componente. Ci sono guasti generici, guasti riguardo alla cassa piena e all’esaurimento di cialde, zucchero e bicchierini.
Quando si verifica un guasto all’assistenza, che è l’unico punto di connessione tra il server e la macchinetta, il server riesce a rilevarlo grazie a un’interruzione dei segnali di ping che vengono mandati dall’assistenza al server in situazione non critica. Dopo un tot di secondi, l’assistenza della macchinetta viene considerata guasta, perciò viene scollegato il subscriber del server associato a essa e viene salvata nel db una richiesta di guasto generico per tale macchinetta.
I guasti generici sono temporizzati.
I guasti vengono risolti manualmente dal tecnico, andando in loco (ferma la macchinetta, risolve e la riaccende). Il tecnico vede le richieste dal sito dell’azienda e, una volta risolta la criticità, la elimina sempre dal sito.

Il test di connessione al broker avviene manualmente quando il tecnico accende/ripara una macchinetta.
Quando il ServerREST ha un problema (si tratta di un caso critico):

- il tecnico non potrà vedere più le richieste, quindi dovrà riparare il ServerREST;
- riparato il ServerREST, si controlleranno le singole macchinette, perché alcune richieste di riparazione potrebbero essere andate perse.

Le macchinette sono programmate per fermarsi in caso di guasto, ma l'assistenza potrebbe ancora mandare conferme (se il guasto è di un'altra componente). Il personale si occuperà quindi di controllare che non si siano perse richieste di riparazione. (es. il tecnico passa a controllare oppure gli impiegati gestiscono le chiamate di avviso di guasti dalle scuole)

Parte gestionale

La parte amministrativa del progetto è stata sviluppata come applicazione web utilizzando il framework ASP.NET con RazorPages. Si tratta di un'applicazione server-side che interagisce con un backend RESTful basato su Apache Spark per la gestione e l'elaborazione dei dati, comunicando con un DBMS MySQL.
L'applicazione consente agli utenti di eseguire operazioni CRUD (Create, Read, Update, Delete) su varie entità, inviare richieste HTTP a un'API Spark e visualizzare i risultati in un'interfaccia utente dinamica e reattiva.

Architettura

L'applicazione è strutturata secondo un'architettura a livelli che include i seguenti componenti:

- Presentazione: Razor Pages per la gestione dell'interfaccia utente.
- Business Logic: Repository pattern per la gestione delle operazioni CRUD.
- Dati: API RESTful esposta tramite un servizio Spark per l'elaborazione e la gestione dei dati.
- Comunicazione: Interazioni HTTP tra l'applicazione web e l'API Spark.
- Autenticazione: Keyclock si occupa dell'autenticazione e autorizzazione degli utenti.
- Portabilità: utilizzo di Docker per la configurazione e l'uso di Keyclock.

Gli utenti possono inviare richieste di creazione, aggiornamento, eliminazione e lettura di varie entità (scuole, macchinette e richieste di guasto) utilizzando l'interfaccia fornita da Razor Pages. Queste richieste vengono tradotte in chiamate HTTP che vengono inviate al backend Spark utilizzando un Client che sfrutta la classe HttpClient, centralizzandone la gestione.
I dati recuperati dalle API Spark vengono visualizzati nelle pagine Razor utilizzando modelli fortemente tipizzati. Inoltre, quando vengono eseguite delle operazioni di post o delete HTTP, ne viene segnalato l'esito all'utente tramite toast. L'applicazione è configurata per utilizzare il protocollo HTTPS, garantendo che tutte le comunicazioni tra i client e il server siano crittografate e sicure, essendo questo protocollo abilitato attraverso l'uso di certificati SSL/TLS che assicurano l'autenticità del server e proteggono i dati trasmessi contro accessi non autorizzati.

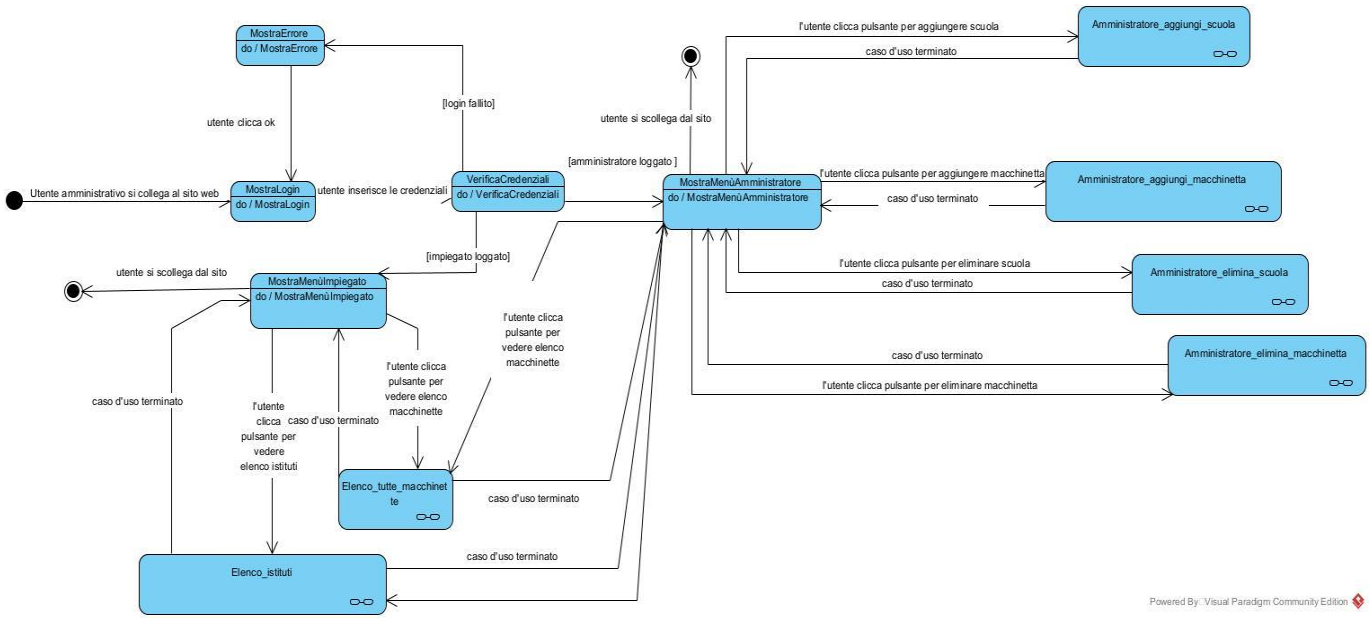
Questo approccio garantisce la coerenza dei dati e facilita la gestione degli errori e delle eccezioni.

Demo (Parte web)

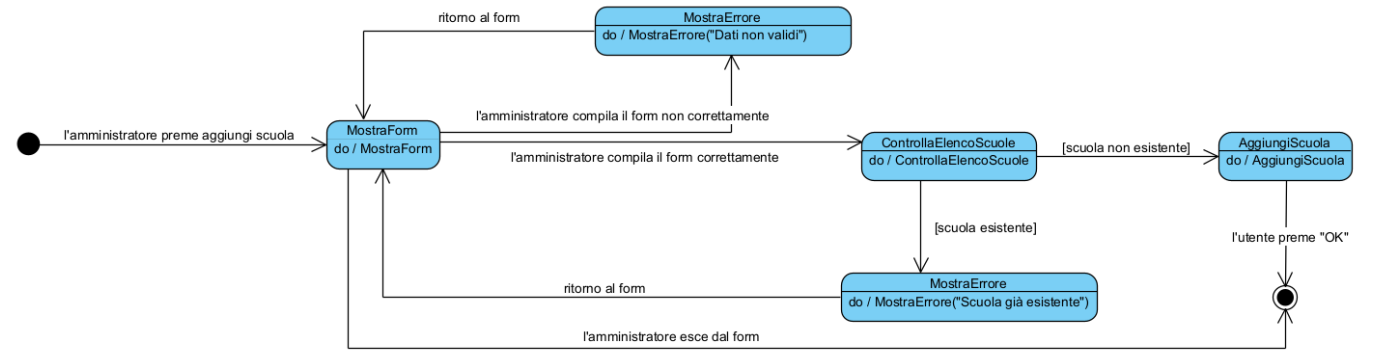
All’avvio l’applicazione si collega a keyclock per eseguire l’autenticazione degli utenti. Il servizio Keyclock è configurato per riconoscere tre tipi di utenti: un amministratore, un impiegato e un tecnico. L’autenticazione avviene tramite inserimento di username e password. Se ci si autentica con successo, l’app riconosce, tramite analisi del token da keyclock, di che ruolo è l’utente e in base a esso viene reindirizzato il menù corretto.
Ci sono tre tipi di menù, uno per l’admin (in cui si possono visualizzare, aggiungere ed eliminare scuole e macchinette), uno per l’impiegato (si possono solo visualizzare scuole e macchinette) e uno per il tecnico (visualizza le richieste di guasto delle macchinette e, una volta risolte in loco, le può eliminare). Nell’elenco delle macchinette di impiegato e amministratore appare anche un simbolo rosso per indicare che la tal macchinetta ha un guasto, cliccandoci sopra sotto la tabella ne verranno visualizzati i dettagli.
Nell’elenco delle scuole, se viene cliccata una riga appare una modale con le macchinette di tal scuola.
Per passare a un altro utente si deve effettuare il logout e, visto che il reindirizzamento non funziona, bisogna chiudere l’applicazione e farla ripartire. Se non si effettua prima il logout, l’utente resta loggato anche dopo il restart dell’applicazione.

State machine diagrams

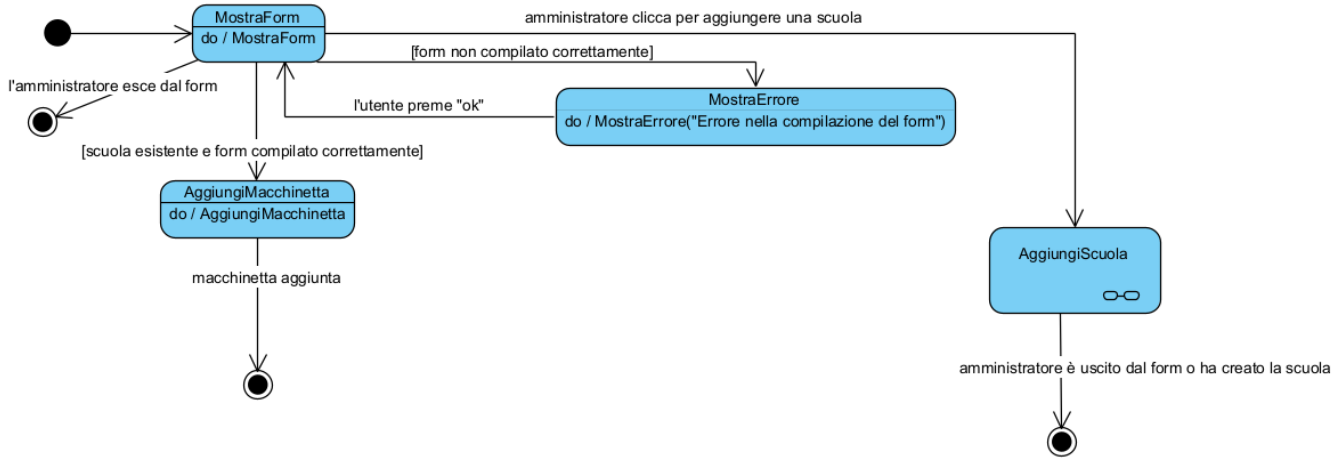
Il lavoratore effettua l'accesso:



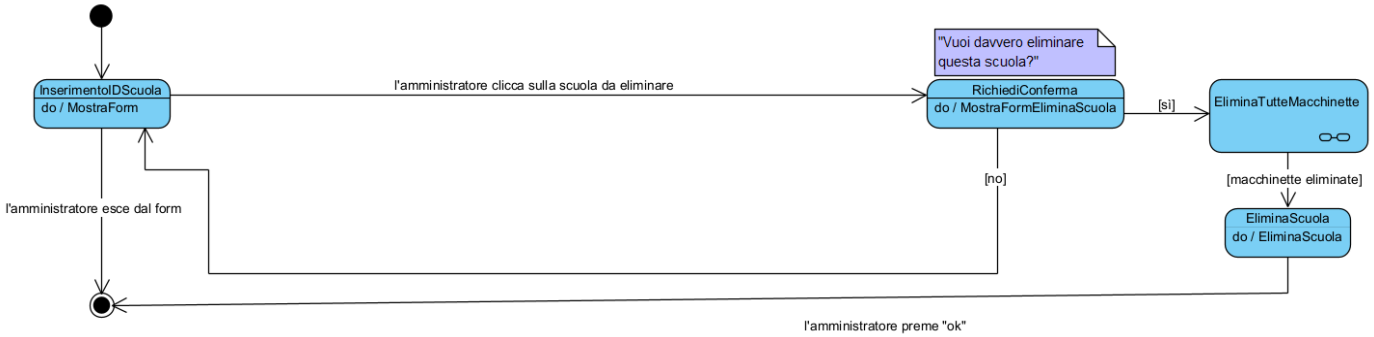
Amministratore aggiunge la scuola:



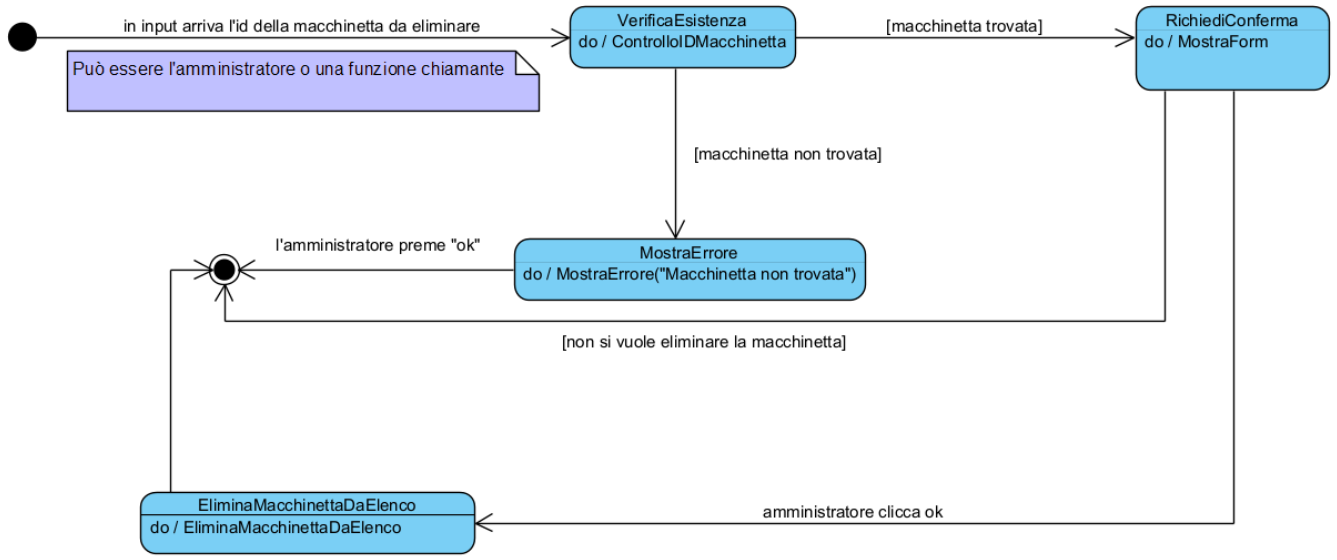
Amministratore aggiunge una macchinetta:



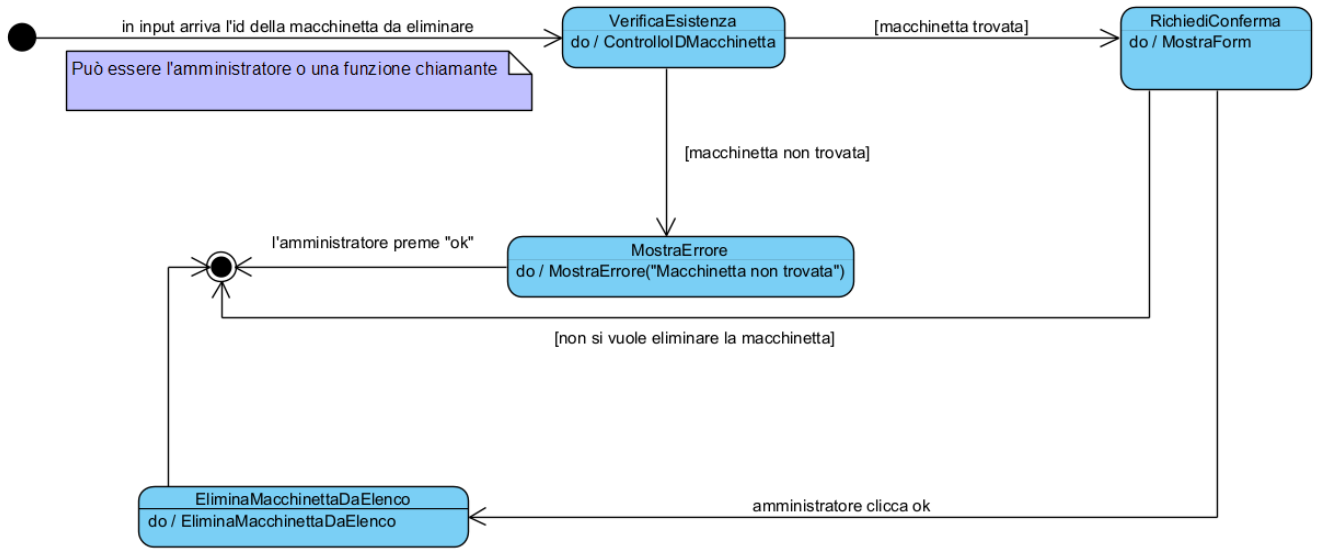
Amministratore elimina una scuola:



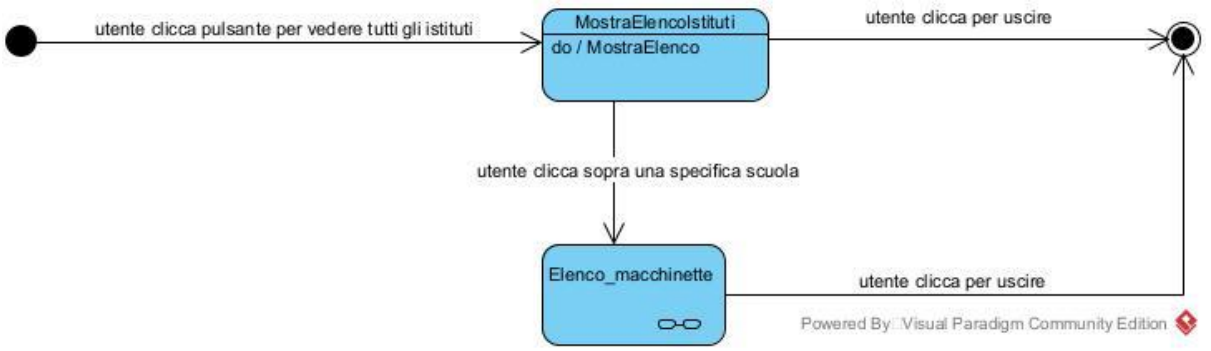
Amministratore elimina una macchinetta:



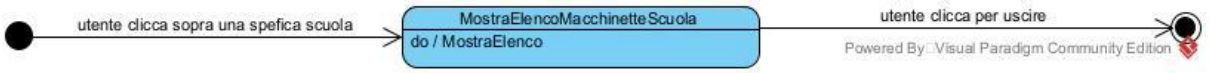
Amministratore elimina tutte le macchinette:



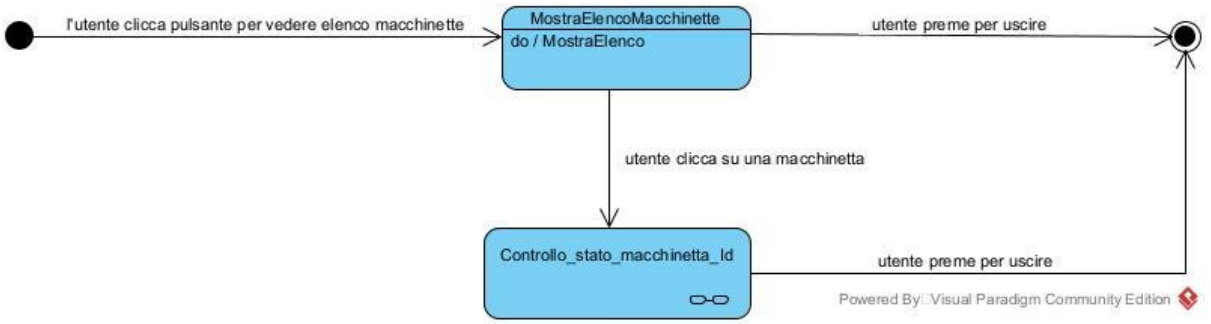
L'utente amministrativo visualizza l'elenco delle scuole:



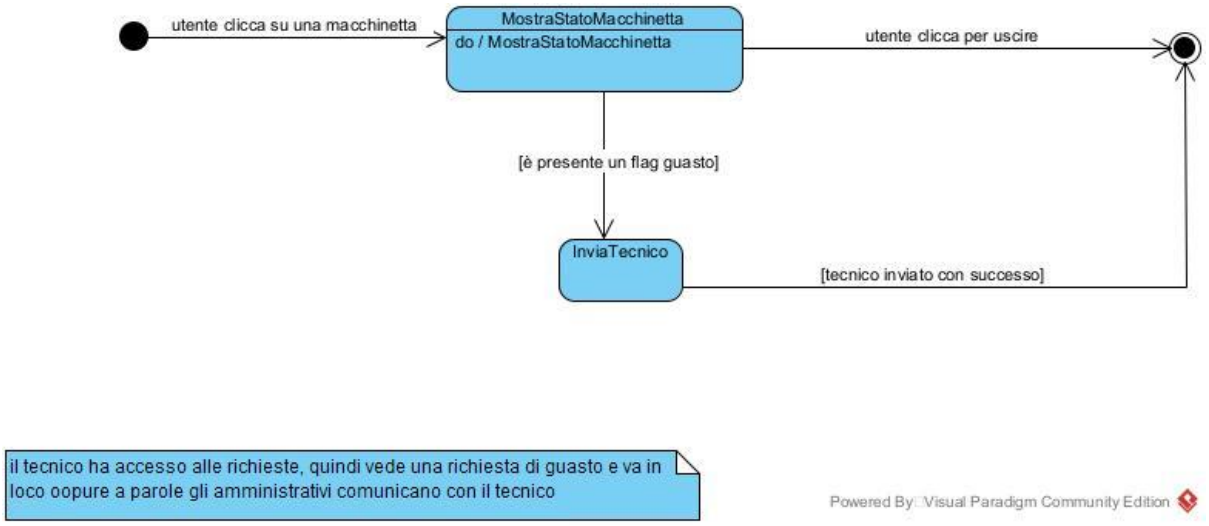
L'utente amministrativo visualizza l'elenco delle macchinette di una scuola:



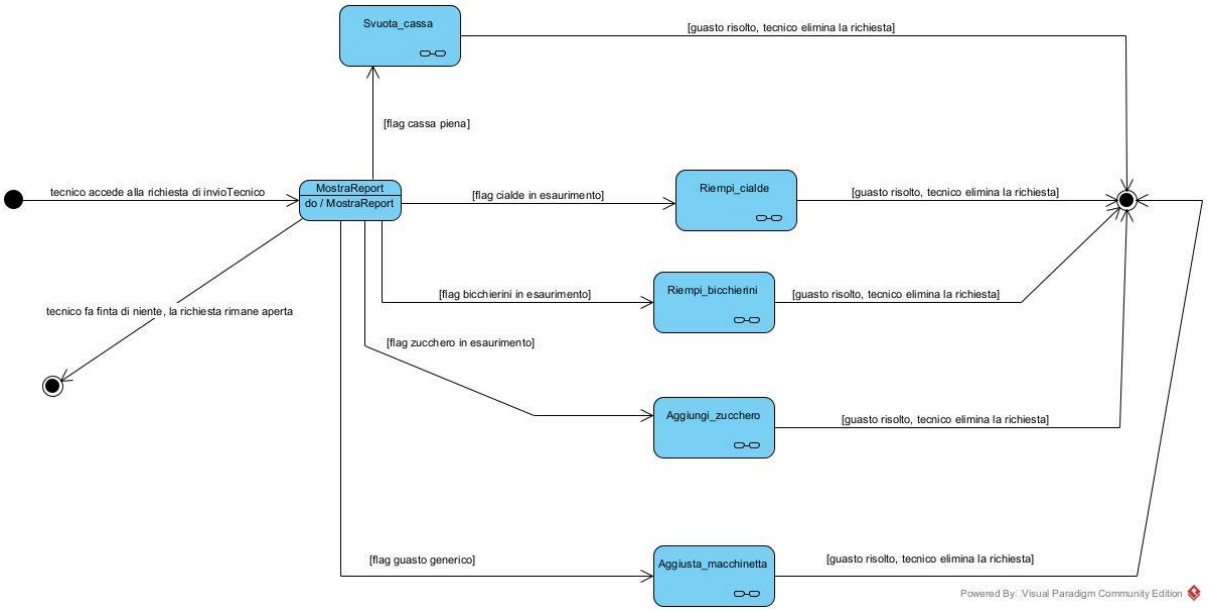
L'utente amministrativo visualizza l'elenco delle macchinette:



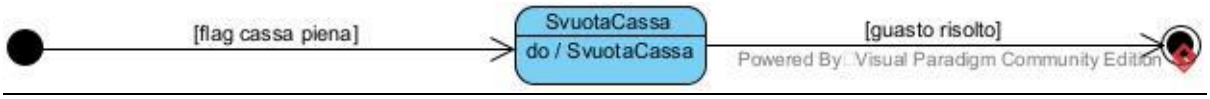
L’utente amministrativo visualizza lo stato di una macchinetta:



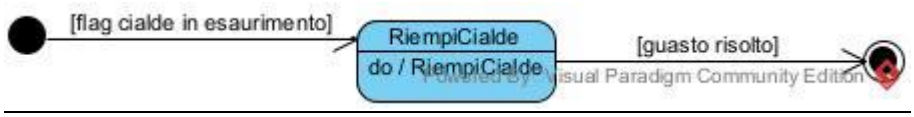
Il tecnico visualizza e risolve le richieste di guasto:



Il tecnico svuota la cassa della macchinetta:

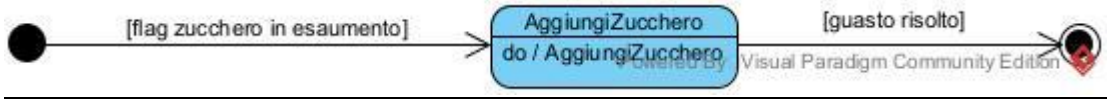


Il tecnico riempie la macchinetta di cialde:

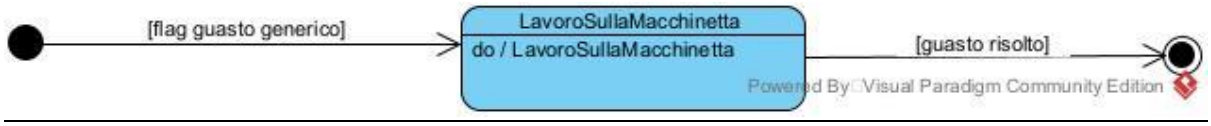


Il tecnico riempie la macchinetta di bicchierini:

Il tecnico riempie la macchinetta di zucchero:

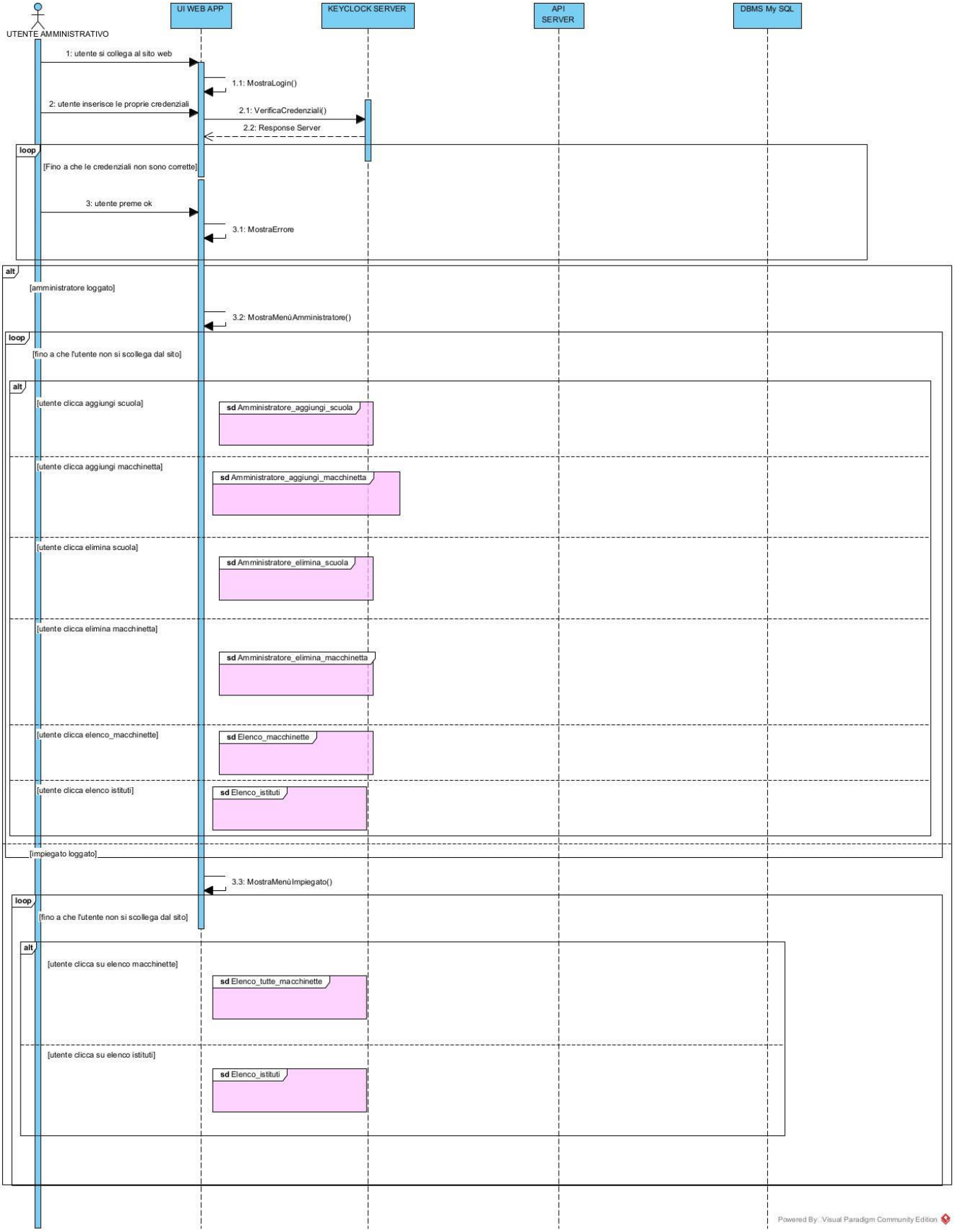


Il tecnico risolve un guasto generico alla macchinetta:

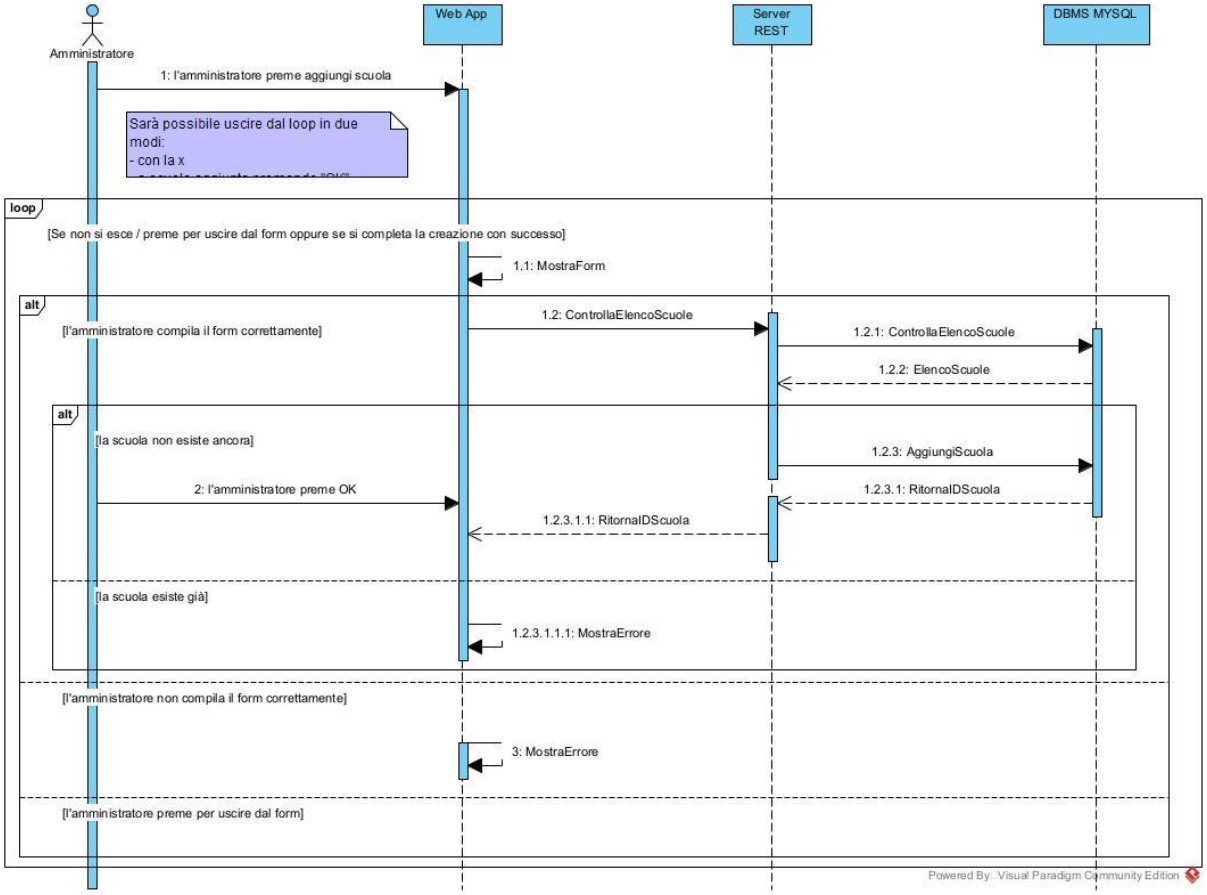


Diagrammi di sequenza

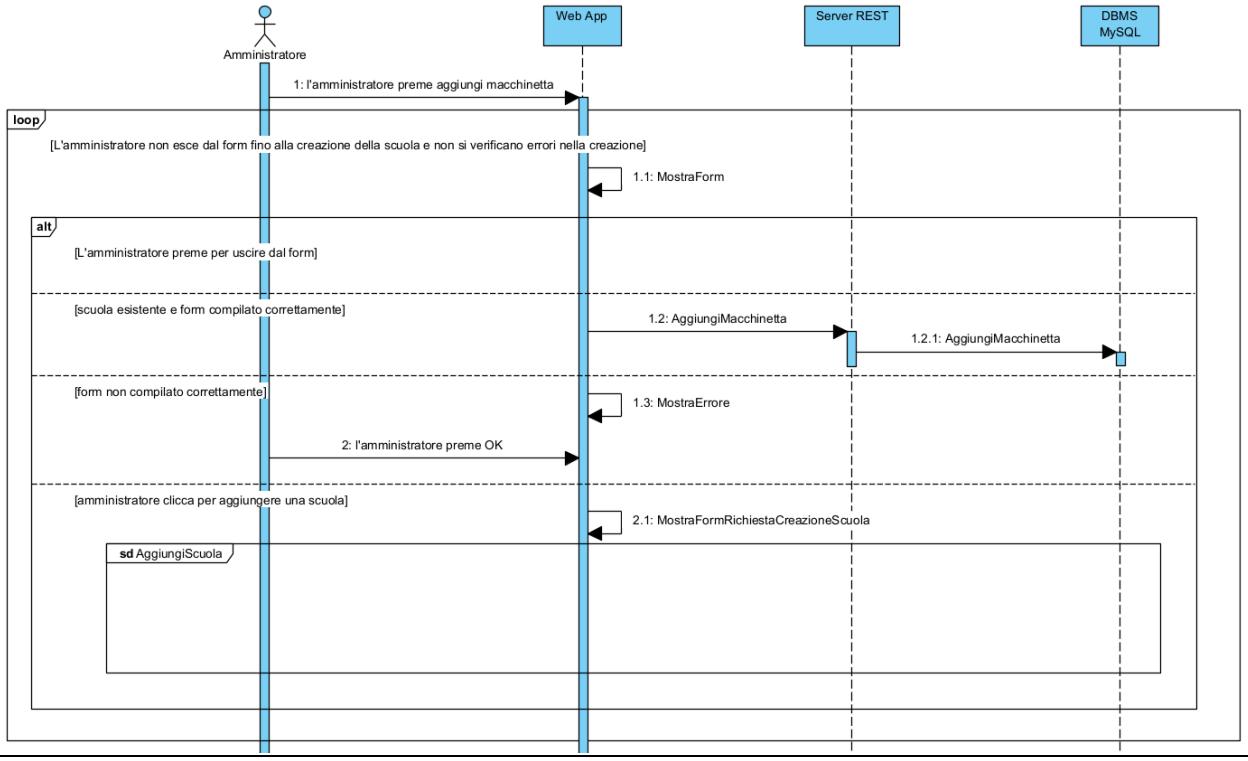
Il lavoratore effettua l’accesso:



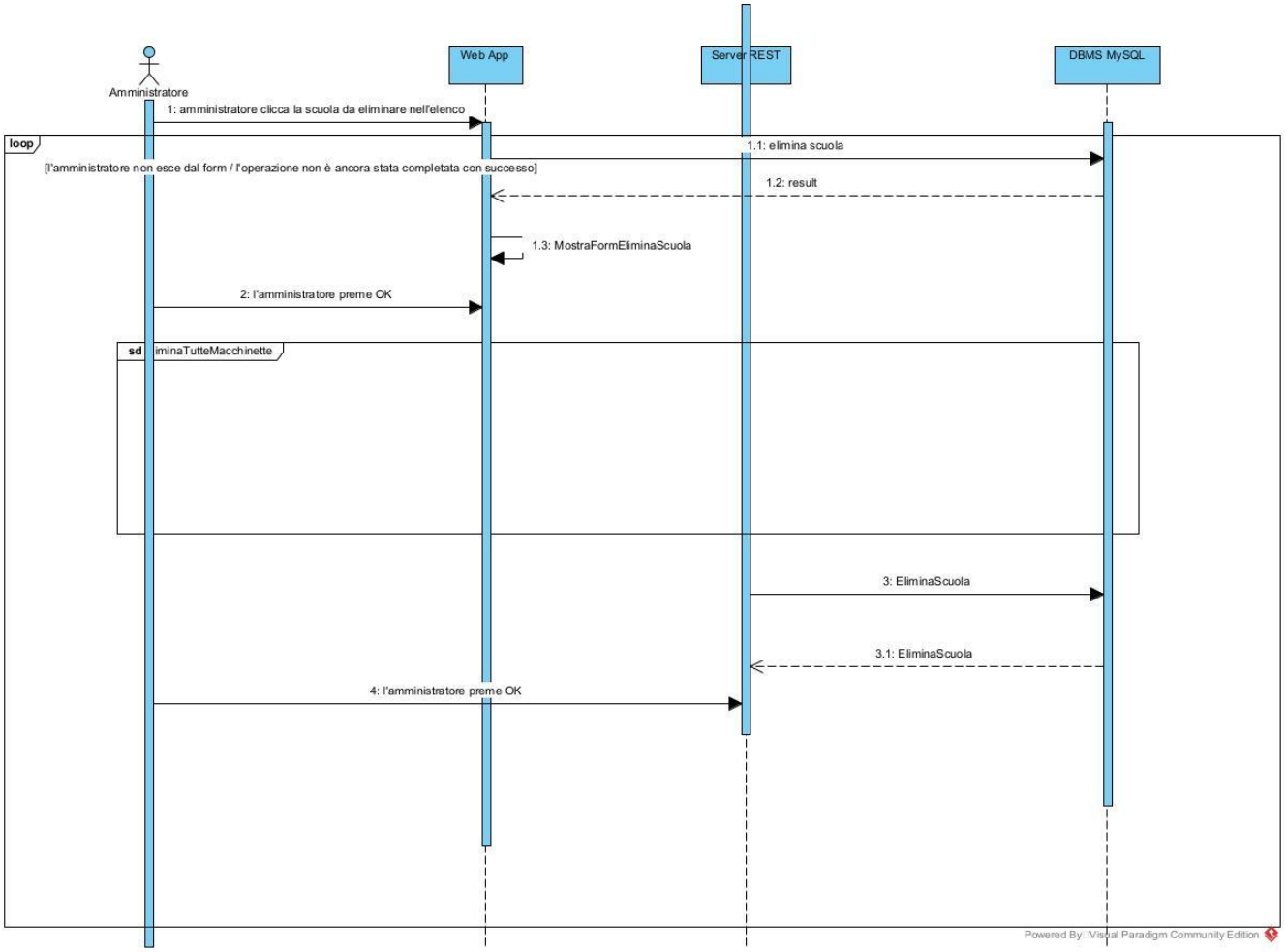
Amministratore aggiunge la scuola:



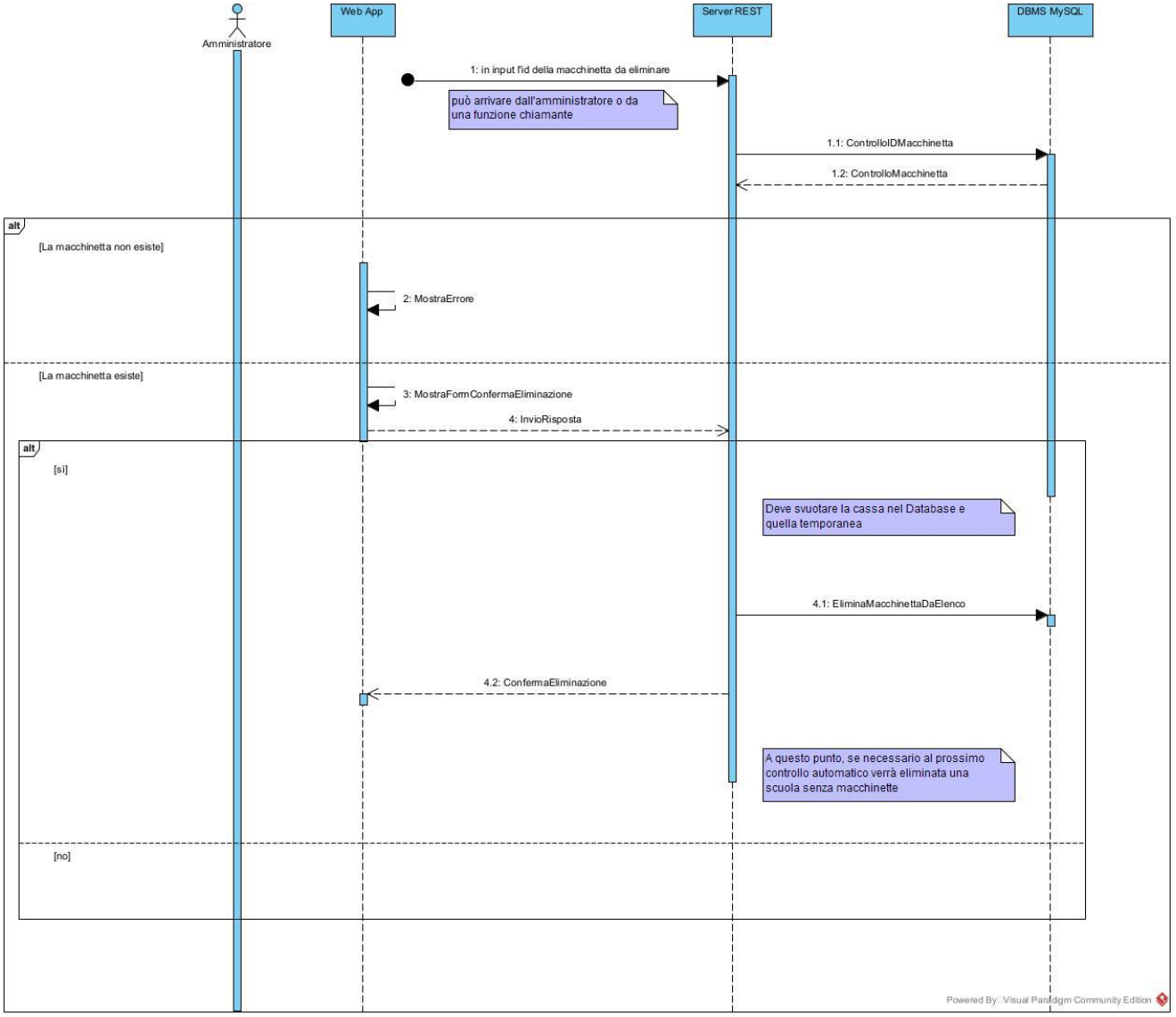
Amministratore aggiunge una macchinetta:



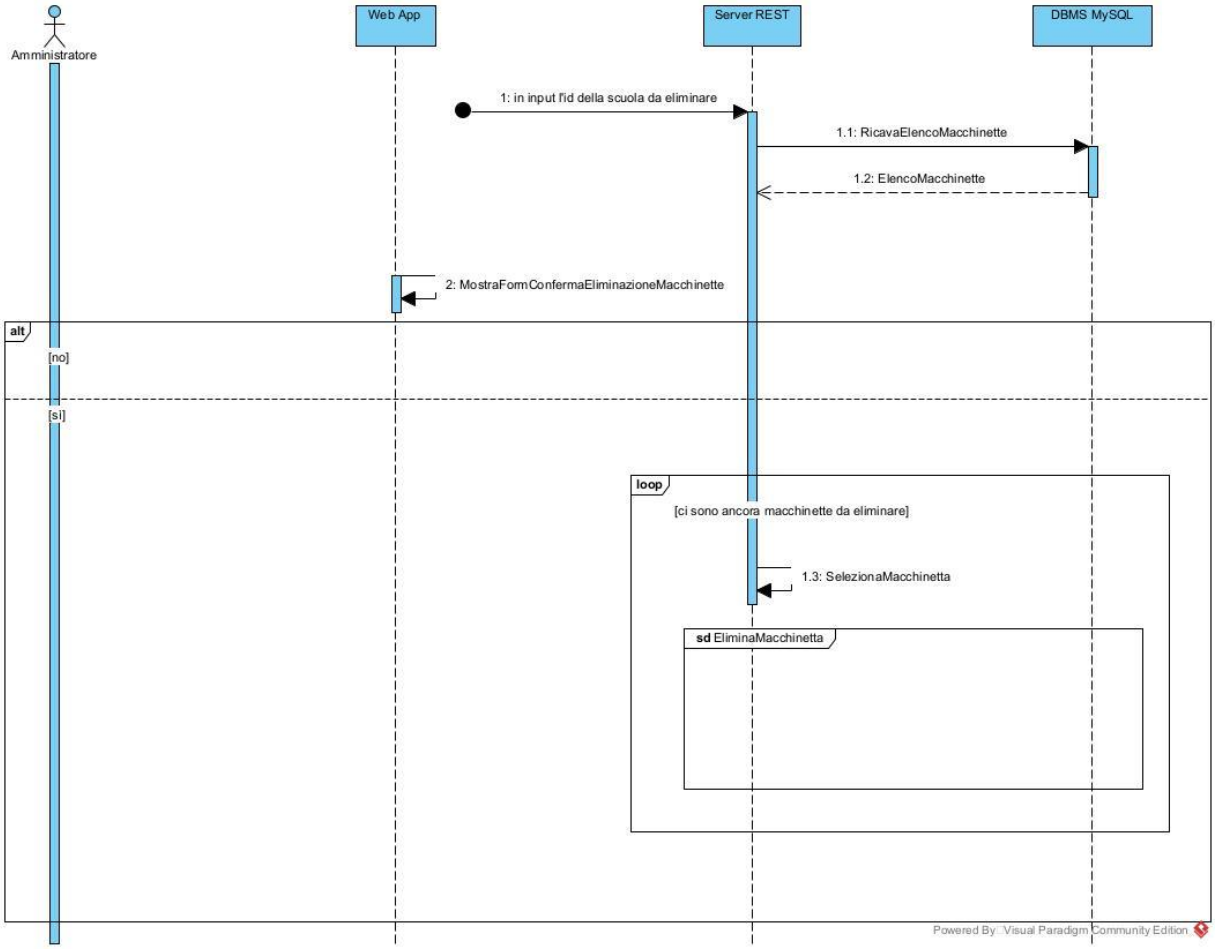
Amministratore elimina scuola:



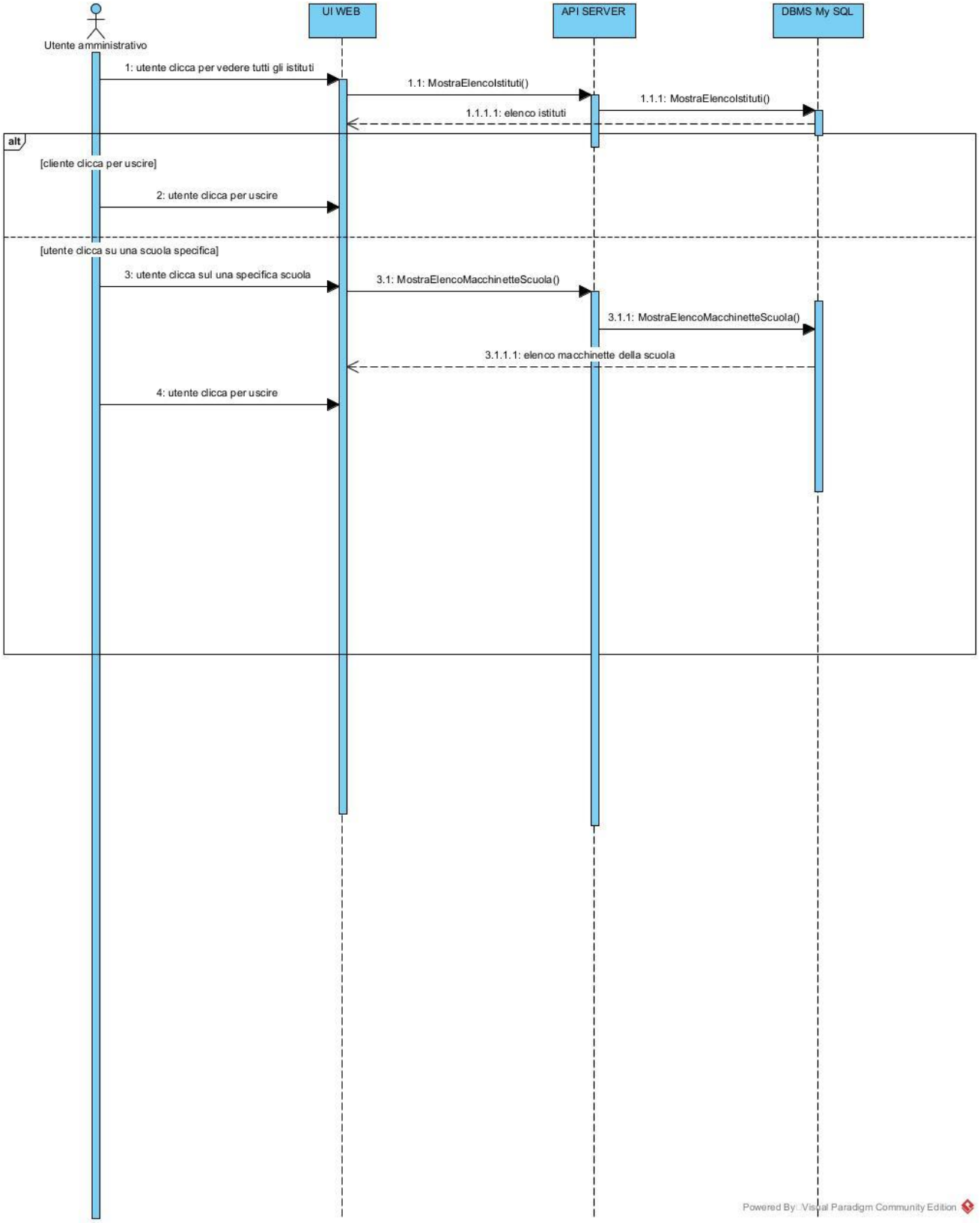
Amministratore elimina macchinetta:



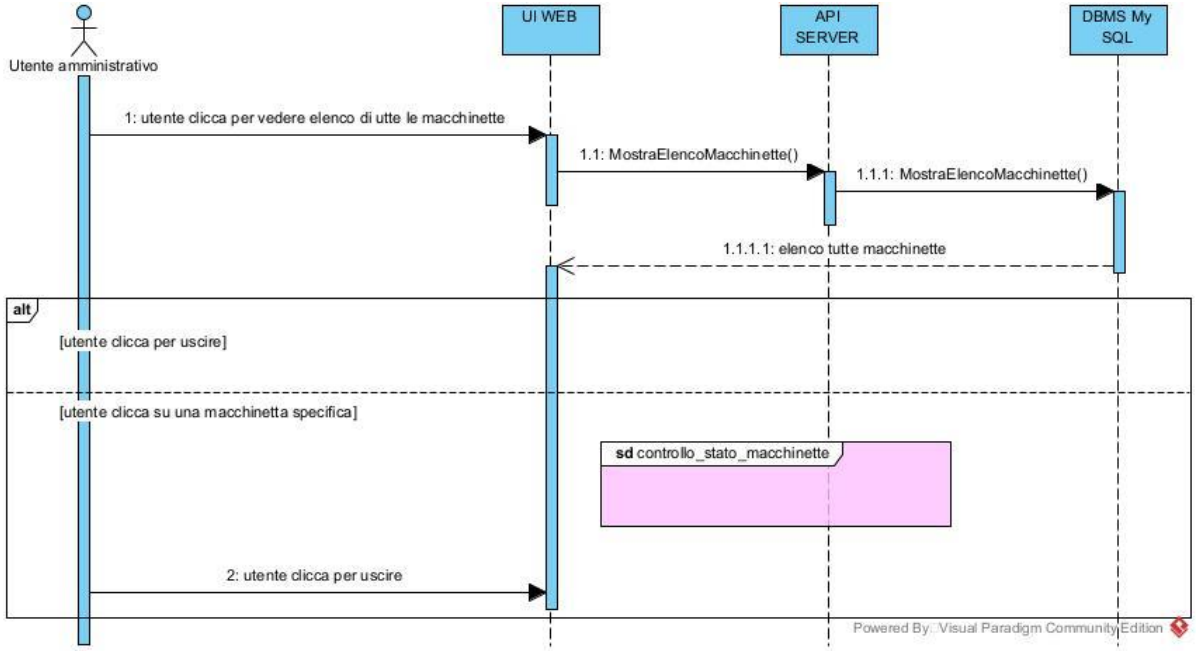
Amministratore elimina tutte le macchinette:



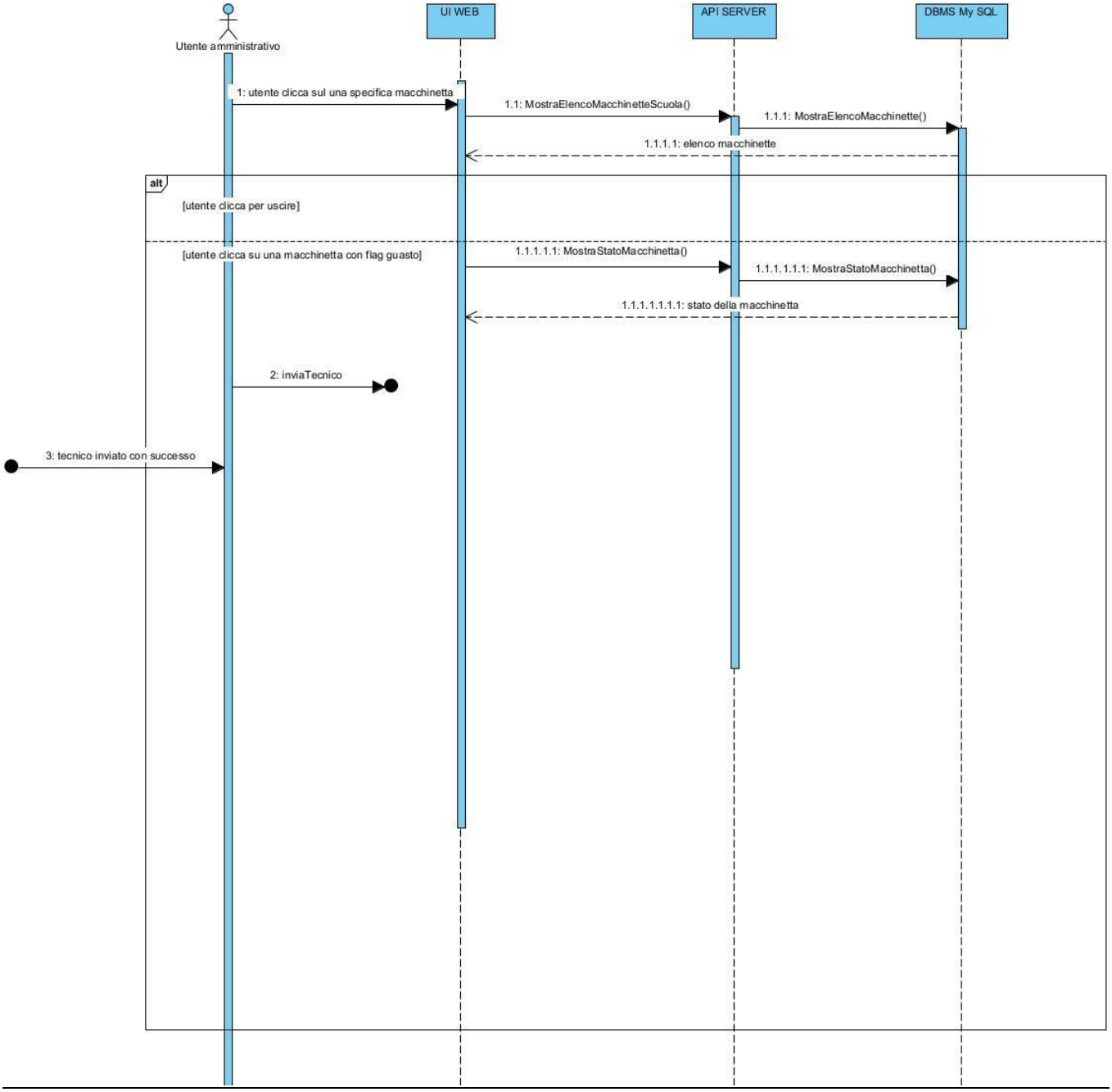
L'utente amministrativo visualizza tutte le scuole ed eventualmente anche le macchinette di una certa scuola:



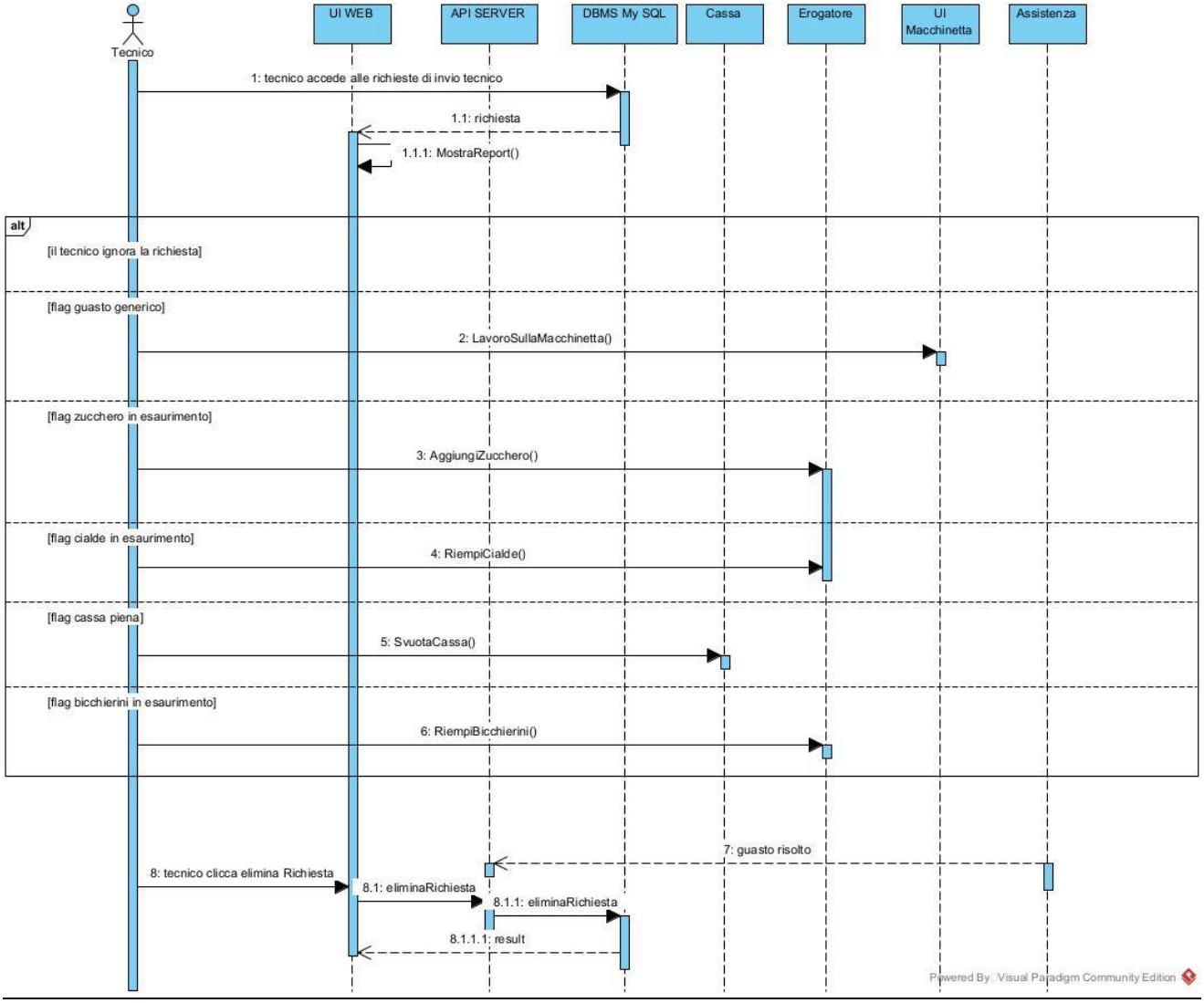
L'utente amministrativo visualizza le macchinette:



L’utente amministrativo visualizza lo stato di una macchinetta:



Il tecnico visualizza e risolve il guasto di una macchinetta:



Template dati

- *Richiesta tecnico*: ci sono diversi flag per problemi di cassa piena, bicchierini in esaurimento, cialde in esaurimento, zucchero in esaurimento, id macchinetta, id scuola;
- *Scuola*: nome, id, città, via, numero civico;
- *Macchinetta*: id, idScuola, cassa, cialde, zucchero, bicchierini, (flag eccezioni, problemi)

Route sul Server REST

- /help: visualizza tutte le route;
- /elenco/istituti: elenco di tutte le scuole;
- /elenco/macchinette: elenco di tutte le macchinette;

- /elenco/macchinette/scuola/:idScuola : elenco macchinette di una scuola;
- /controllo/macchinetta/:idMacchinetta : elenco tutti i parametri della macchinetta che ha idMacchinetta;
- /inviaTecnico : post dei dati della richiesta per richiedere il tecnico;
- /riceviRichieste: get per prendere la richiesta più recente di invia tecnico;
- /add/scuola : post per il form per l'aggiunta della scuola;
- /add/:idScuola/macchinetta: post per il form per l'aggiunta della macchinetta;
- /delete/scuola/:idScuola :elimina la scuola che ha idScuola;
- /delete/macchinetta/:idMacchinetta: elimina la macchinetta che ha idMacchinetta;

(elimina tutte macchinette di una scuola lo si fa direttamente all'interno della route /delete/scuola/:idScuola, non c'è bisogno di un'altra route)

Istruzioni per importare il database MySql

1. creare un db in mysql chiamato *ProgettoMacchinetteParteAmministrativa* utilizzando il server MySQL in esecuzione sulla porta *3306*, user *root* e password zLfVSEAz8rKaUi;
2. importare il file .sql di backup inserito nella cartella backupmysql

Istruzioni per avviare keycloak e accedere al sito (web app)

1. far partire il Server REST su Eclipse e lasciare in background;
2. aprire docker desktop;
3. aprire terminale windows;
4. cd ".....\Progetto reti 2\progetto-reti-2\VisualStudio\AziendaMacchinetteBevande\" ;
5. scivere ed eseguire il comando *docker-compose up* ;
6. lasciare in background (quando raggiunge l'avviso di non usare quella configurazione in fase di produzione ha finito di caricarsi, lasciarlo in background senza chiudere il terminale);
7. far partire l'ide di visual studio;
8. aprire AziendaMacchinetteBevande.sln (tasto per aprire progetto o soluzione) che si trova dentro la cartella VisualStudio/AzienaMacchinetteBevande;
9. far eseguire la soluzione in modalità https (quindi premere play con https);
10. Si aprirà la pagina di login di keyclock, fare il login con uno degli utenti.

Ecco le credenziali:

Amministratore
user: amministratore-user
password: ciaoatutti4

Impiegato:
user: impiegato-user
password: ciao99:-)

Tecnico:
user: tecnico-user
password: chiaveInglese

Si verrà reindirizzati al menù correlato. Tasti per fare logout o ritornare alla pagina principale del menù sono in alto nella barra.

ATTENZIONE
A causa di problemi di gestione di backup di Keyclock (problemi interni a Keyclock relativi al suo uso con Docker), Docker, per creare il container, recupera un file di backup parziale (l'unico che si può creare dalla admin console) il quale contiene i clienti (l'app web) e i ruoli (admin, employee e technician) MA non gli user. La documentazione di keyclock non è aggiornata, perciò non si è riuscito a fare un backup totale con anche gli utenti causa errori interni con il db che gestisce direttamente Keyclock; quindi per l'esecuzione della web app è necessario o accedere alla admin console e aggiungere gli utenti oppure attendere l'esecuzione sul pc della studentessa.

Test

I test sono stati effettuati sia per validazione sia per verifica (utilizzando JUnit).
Molti dei test JUnit sono prove dei possibili messaggi che un microservizio potrebbe ricevere dal broker di Mosquitto.
Quando non sono stati testati direttamente con la JUnit, comunque, sono stati testati per validazione utilizzando i publisher e subscriber da terminale, in modo da isolare altri problemi che si sarebbero potuti verificare legati ad altre componenti del codice.

Istruzioni e demo (Macchinetta)

Per prima cosa bisogna assicurarsi di rispettare tutti i requisiti precedentemente elencati (versione della jdk, mosquitto, ecc…).

La connessione con Mosquitto avviene nelle singole componenti all'avvio di ciascuna:

- ogni componente può loggarsi con uno di più account possibili:
  - cassa1
  - erogatore1
  - interfaccia_utente1
  - assistenza1
- abbiamo utilizzato tre certificati con corrispondente chiave:
  - quello della CA
  - quello del broker
  - uno condiviso dai client

  In un'applicazione commerciale, potrebbero essere necessari più certificati in base alla struttura e alle necessità. Inoltre, le chiavi andrebbero tenute in un posto sicuro;

La configurazione corretta del broker è indicata nei file contenuti nella cartella "Configurazione MQTT e TLS" e contiene i certificati per tutte le componenti, una copia dei file mosquitto.conf, mosquitto.pwd, acl_file.txt e tutto il necessario.
Le path nel programma si basano su tale cartella e sulla sua posizione rispetto alla cartella Eclipse, la quale contiene i progetti Maven del ServerREST (5676579) e della macchinetta (macchinettaPISSIR)

Per avviare manualmente le singole componenti della macchinetta senza incorrere subito in guasti simulati, è <u>fortemente consigliato</u> accendere le componenti nel seguente ordine:

1) DBLocalCreator

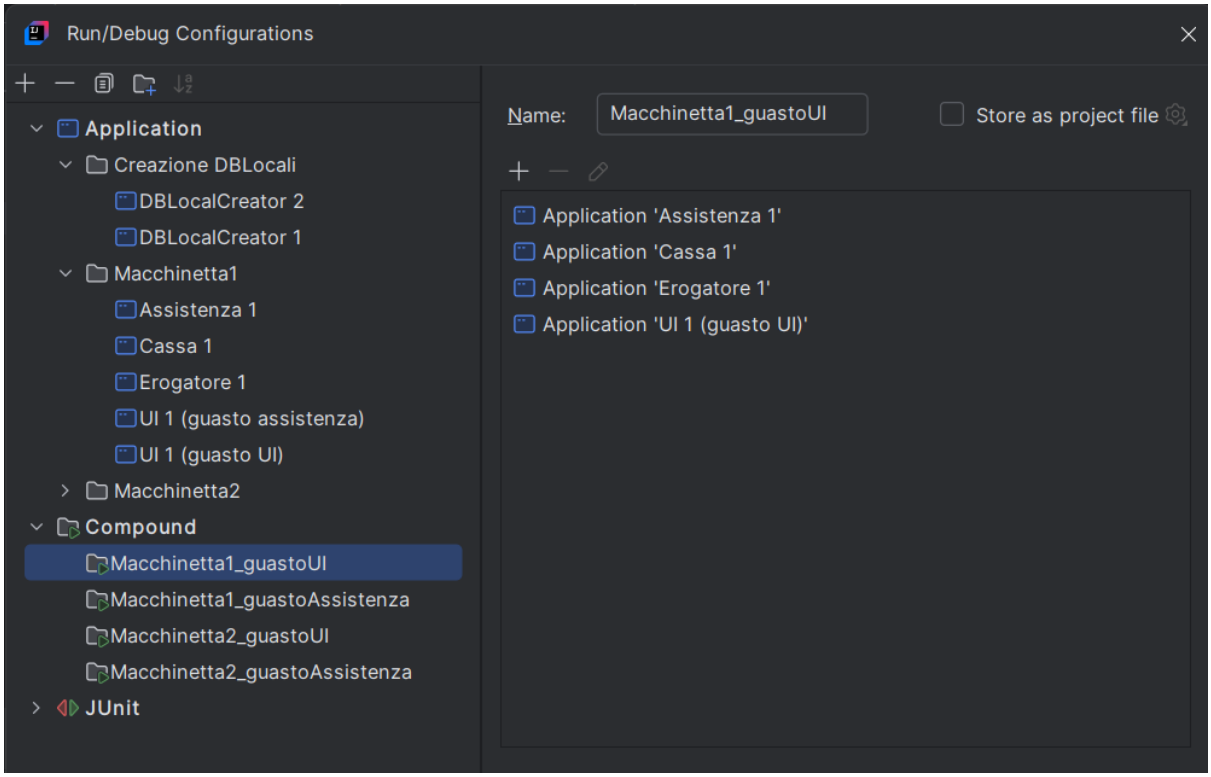
2) Configurazione per creare una macchinetta (se le componenti sono eseguite singolarmente, l'ordine consigliato è il seguente, altrimenti la configurazione eseguirà tutto insieme):
    a) Cassa ed Erogatore
    b) Assistenza
    c) Interfaccia Utente
3) Server REST
4) Nel caso, parte gestionale

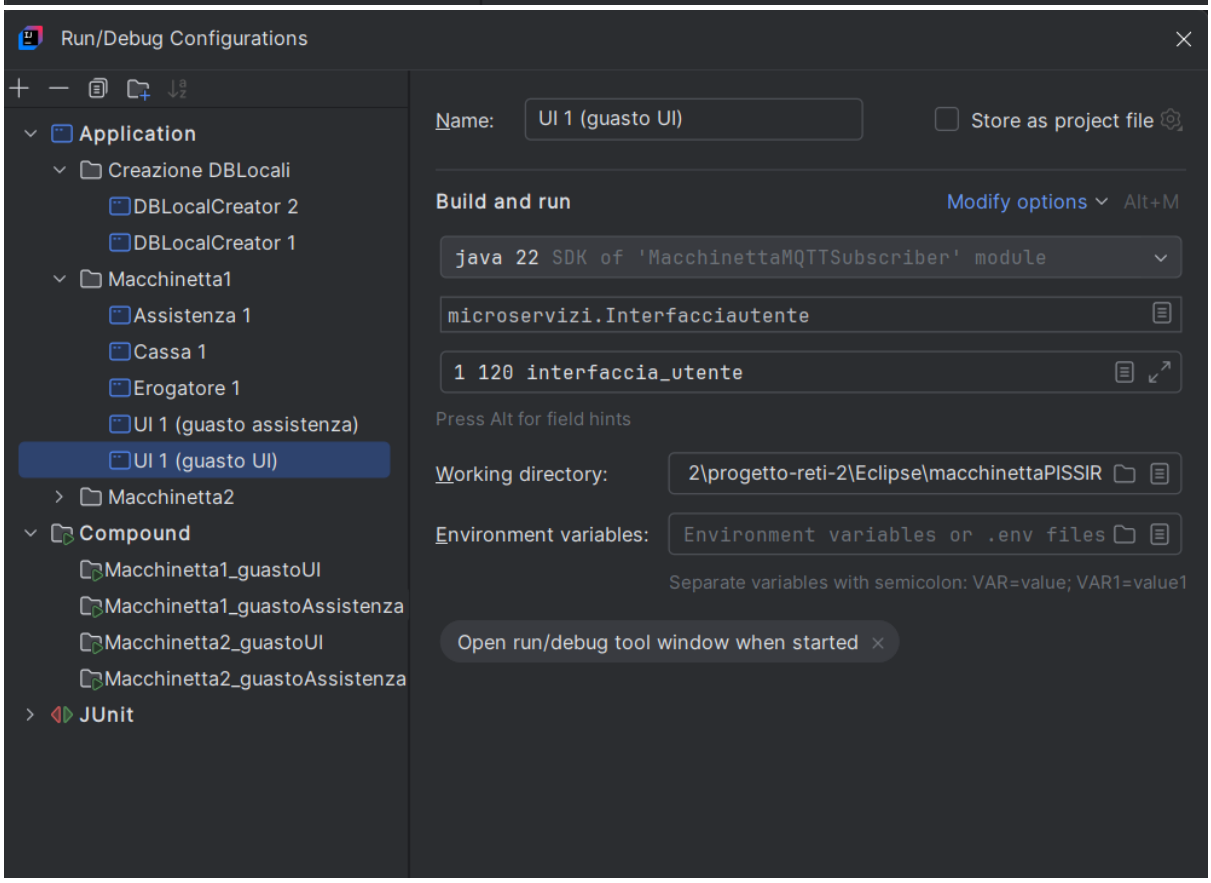
L'avvio delle macchinette è manuale: dopo aver creato una configurazione di tipo Application per eseguire il DBLocalCreator, ci siamo affidati al tipo di configurazione Compound di Intellij per creare contemporaneamente tutti i microservizi della macchinetta (cassa, erogatore, assistenza, UI).

Ogni singola applicazione che la compone deve tenere conto degli input richiesti al momento del lancio del main:

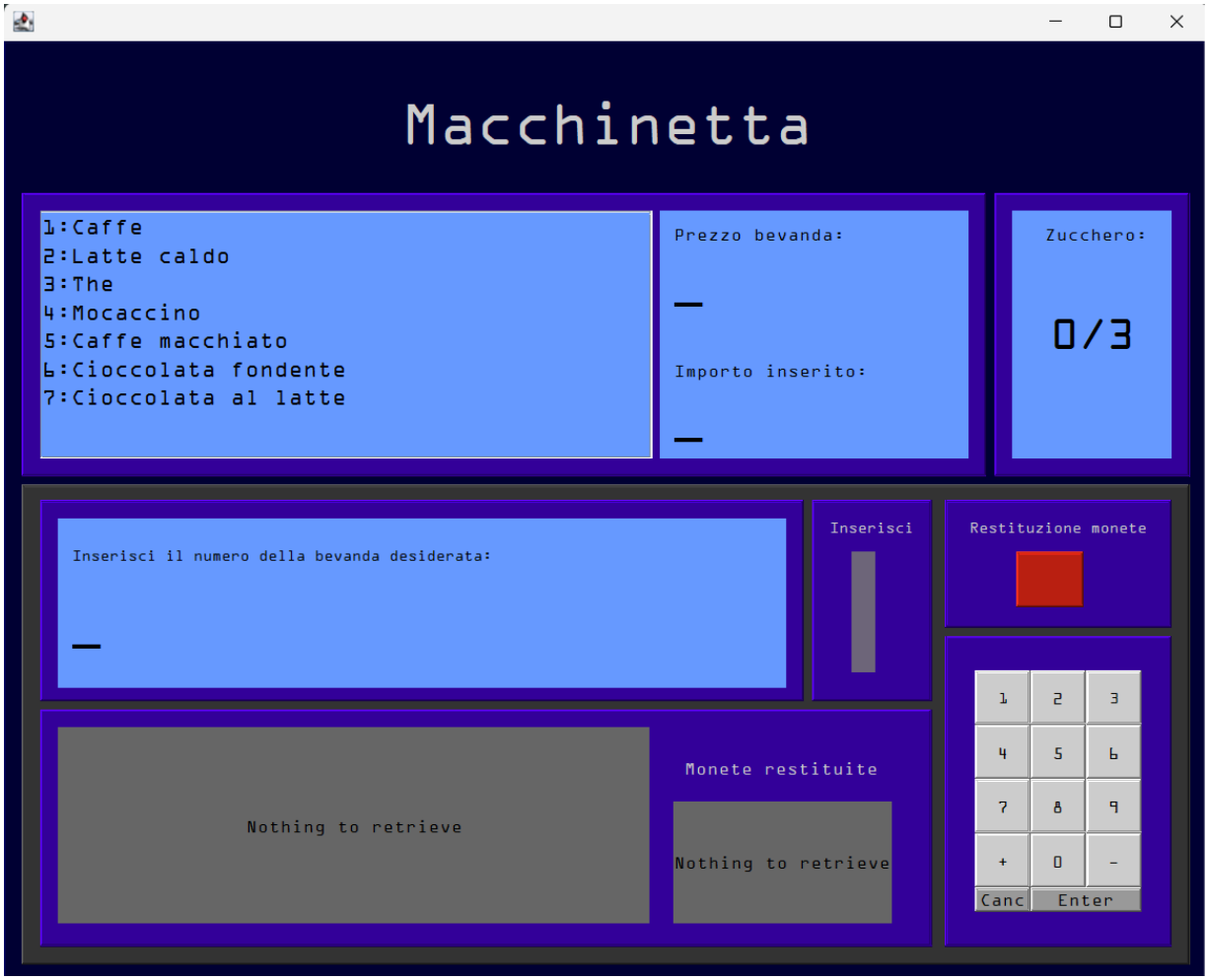
- Assistenza, Cassa ed Erogatore richiedono l'id della macchinetta;
- Interfaccia Utente:
    - l'id della macchinetta;
    - i secondi dalla partenza del programma per generare un guasto generico;
    - "interfaccia_utente" per indicare su quale componente generare tale guasto, non generando niente in caso di inserimento di stringhe al di fuori di quelle riconosciute dalle singole funzioni di guasto;

Sono necessarie configurazioni per ogni macchinetta.
Nel nostro caso, le configurazioni con cui abbiamo lavorato sono le seguenti:


Run/Debug Configurations
Application
Creazione DBLocali
DBLocalCreator 2
DBLocalCreator 1
Macchinetta1
Assistenza 1
Cassa 1
Erogatore 1
UI 1 (guasto assistenza)
UI 1 (guasto UI)
Macchinetta2
Compound
Macchinetta1_guastoUI
Macchinetta1_guastoAssistenza
Macchinetta2_guastoUI
Macchinetta2_guastoAssistenza
JUnit
Name:
Macchinetta1_guastoUI
Store as project file
Application 'Assistenza 1'
Application 'Cassa 1'
Application 'Erogatore 1'
Application 'UI 1 (guasto UI)'


Run/Debug Configurations
Application
Creazione DBLocali
DBLocalCreator 2
DBLocalCreator 1
Macchinetta1
Assistenza 1
Cassa 1
Erogatore 1
UI 1 (guasto assistenza)
UI 1 (guasto UI)
Macchinetta2
Compound
Macchinetta1_guastoUI
Macchinetta1_guastoAssistenza
Macchinetta2_guastoUI
Macchinetta2_guastoAssistenza
JUnit
Name:
UI 1 (guasto UI)
Store as project file
Build and run
Modify options
Alt+M
java 22 SDK of 'MacchinettaMQTTSubscriber' module
microservizi.Interfacciautente
1 120 interfaccia_utente
Press Alt for field hints
Working directory:
2\progetto-reti-2\Eclipse\macchinettaPISSIR
Environment variables:
Environment variables or .env files
Separate variables with semicolon: VAR=value; VAR1=value1
Open run/debug tool window when started

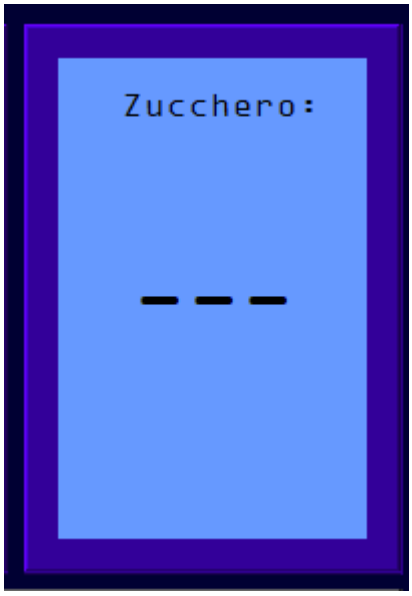
All’avvio, se non ci sono problemi (es. mancanza di bicchierini, troppe poche cialde per erogare qualsiasi bevanda, cassa piena o altri guasti), comparirà la seguente schermata:



È composta da bottoni e schermi su cui visualizzare il risultato delle elaborazioni.
In particolare, premendo:

- Inserisci: è possibile inserire delle monete, in base ai tipi di monete proposte dalla UI;
- Restituzione monete: darà indietro all’utente il contenuto della cassa temporanea;
- Monete restituite: si potrà ritirare il resto erogato dalla macchinetta;
- Tastierino numerico:
    - numeri da 0 a 9: servono a scegliere la bevanda (tra quelle disponibili, visualizzate nello schermo in alto a sinistra) e verranno scritti sullo schermo “Inserisci il numero della bevanda desiderata”;
    - Canc: serve a cancellare l’ultima cifra inserita;
    - + e -: servono a regolare la quantità di zucchero che si desidera mettere nella bevanda, quando lo zucchero è disponibile (comparirà sullo schermo “Zucchero” in alto a destra);

    - Enter: prende il numero della bevanda inserito, la quantità di zucchero stabilita e comunica i dati alla cassa. Se la cassa temporanea contiene un importo sufficiente, l'erogazione avrà già inizio nel mentre. Altrimenti, verrà solo mostrato il prezzo della bevanda. Il contenuto della cassa temporanea e il prezzo della bevanda selezionata sono mostrati nello schermo apposito, appena a sinistra di quello dello zucchero;
- Bevanda erogata (tasto grande appena a sinistra di Monete restituite): l'utente potrà ritirare la propria bevanda, una volta pronta.

Prima di finire l'utilizzo della macchinetta (chiusura della finestra) verrà richiesta una conferma.
In caso di guasto improvviso, la gui si disabiliterà; quando il tecnico avrà risolto il guasto, la macchinetta verrà riavviata.

Quando lo zucchero è in esaurimento la macchinetta non si guasta, semplicemente smetterà di erogare lo zucchero. Ecco come sarà lo zucchero disabilitato: